МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ТУЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт прикладной математики и компьютерных наук

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
по дисциплине «ПРОГРАММИРОВАНИЕ»

_____

(тема курсовой работы)

Студент гр. _____ _____ _____

(индекс группы) (подпись и дата) (инициалы и фамилия)

Руководитель _____ _____ _____

(должность и ученая степень) (подпись и дата) (инициалы и фамилия)

ТУЛА 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 6
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 8
1.1 Проблематика и актуальность 8
1.2 Анализ существующих решений 9
1.3 Цель и задачи проекта 10
2 ТЕХНИЧЕСКОЕ ЗАДАНИЕ 10
2.1 Назначение разработки 10
2.2 Функциональные требования 10
2.2.1 Основные функции 10
2.2.2 Дополнительные функции 11
2.3 Требования к интерфейсу пользователя 11
2.3.1 Общие требования 11
2.3.2 Структура интерфейса 12
2.4 Требования к надежности и безопасности 12
2.5 Требования к эргономике 12
2.6 Стадии и этапы разработки 13
3 МОДЕЛИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММЫ НА ОСНОВЕ ПРИНЦИПОВ ООП 13
3.1 Архитектура программного обеспечения 13
3.2 Проектирование классов 14
3.2.1 Класс Document 14
3.2.2 Класс DocumentStorage 15
3.2.3 Класс GUIWindow (неявный) 15
3.3 Диаграмма классов 16
3.4 Алгоритмы и методы 16
3.4.1 Алгоритм триграммного поиска 16
3.4.2 Алгоритм сериализации данных 17
3.5 Проектирование базы данных 18
4 ОПИСАНИЕ ПРОГРАММЫ 18
4.1 Функциональное назначение 18

4.2 Описание логической структуры 19
4.2.1 Модульная структура 19
4.2.2 Поток данных 20
4.2.3 Структура данных в памяти 21
4.3 Входные данные 21
4.3.1 Типы входных данных 21
4.3.2 Источники входных данных 22
4.4 Выходные данные 22
4.4.1 Форматы выходных данных 22
4.4.2 Типы сообщений 22
4.5 Описание модулей программы 22
4.5.1 Модуль Document 22
4.5.2 Модуль DocumentStorage 23
4.5.3 Модуль графического интерфейса 23
4.6 Используемые технологии и библиотеки 24
4.6.1 Язык программирования и стандарт 24
4.6.2 Платформенные API 24
4.6.3 Внешние зависимости 24
4.6.4 Кодировки и локализация 25
5 ИНСТРУКЦИЯ ПО ЭКСПЛУАТАЦИИ ПРОГРАММЫ 25
5.1 Назначение программы 25
5.2 Системные требования 25
5.2.1 Минимальные требования 25
5.2.2 Рекомендуемые требования 25
5.2.3 Зависимости 25
5.3 Установка и запуск 26
5.3.1 Установка программы 26
5.3.2 Запуск программы 26
5.4. Интерфейс пользователя. 27
5.5 Работа с программой 35
5.5.1 Добавление документа 35
5.5.2 Редактирование документа 35

5.5.3 Удаление документа 36
5.5.4 Поиск документов 36
5.5.5 Загрузка данных из файла 37
5.5.6 Сохранение данных в файл 37
5.5.7 Очистка всех документов 37
5.6 Описание контрольного примера 37
5.6.1 Подготовка тестовых данных 37
5.6.2 Последовательность тестирования 38
5.7 Возможные ошибки и способы их устранения 39
5.7.1 Ошибки при запуске программы 39
5.7.2 Ошибки при работе с документами 39
5.7.3 Ошибки при загрузке/сохранении 39
5.7.4 Общие рекомендации 40
6 ТЕСТИРОВАНИЕ И ОТЛАДКА ПРОГРАММЫ 40
6.1 Методология тестирования 40
6.1.1 Виды тестирования 40
6.1.2 Инструменты тестирования 40
6.2 Тестовые сценарии 40
6.2.1 Сценарий 1: Работа с документами 40
6.2.2 Сценарий 2: Поиск документов 41
6.2.3 Сценарий 3: Работа с файлами 41
6.3 Результаты тестирования 41
6.3.1 Функциональное тестирование 41
6.3.2 Нефункциональное тестирование 42
6.3.3 Найденные и исправленные ошибки 42
6.4 Отладка программы 42
6.4.1 Используемые методы отладки 42
6.4.2 Типичные проблемы и решения 42
7 ЗАКЛЮЧЕНИЕ 43
7.1 Достигнутые результаты 43
7.2 Соответствие требованиям 43
7.3 Научно-практическая значимость 43

7.4 Перспективы развития 43
7.5 Выводы 44
8 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ 44
8.1 Нормативные документы 44
8.2 Книги и учебные пособия 45
8.3 Статьи и научные публикации 45
8.4 Интернет-ресурсы 45
8.5 Программное обеспечение 45
ПРИЛОЖЕНИЯ 46
Приложение А. Листинг программы 46
Приложение Б. Диаграмма последовательностей 75
Приложение В. Тестовые данные 75
Приложение Г. Руководство по установке 76

ВВЕДЕНИЕ

В современном строительном секторе управление проектной документацией представляет собой сложную и многоуровневую задачу, имеющую критическое значение для успешной реализации строительных проектов. С ростом объемов строительства и усложнением проектов традиционные методы хранения и обработки документации на бумажных носителях становятся все менее эффективными и более затратными.

Актуальность проблемы организации эффективного хранения проектной документации обусловлена несколькими ключевыми факторами. Во-первых, строительные проекты сопровождаются значительными объемами документации, включающей чертежи, технические задания, сметы, разрешительные документы и другую важную информацию. Во-вторых, необходимость обеспечения быстрого доступа к документам для различных специалистов (архитекторов, инженеров, прорабов, сметчиков) требует создания централизованной системы управления. В-третьих, требования к сохранности и конфиденциальности проектной документации диктуют необходимость внедрения специализированных решений.

Традиционные методы организации хранения строительной документации, основанные на бумажных архивах и локальных электронных хранилищах, имеют ряд существенных недостатков. К ним относятся: высокий риск потери или повреждения документов, сложности в поиске нужной информации, ограниченный одновременный доступ для нескольких пользователей, отсутствие системы контроля версий и истории изменений, а также значительные затраты на физическое хранение и обслуживание архивов.

Развитие информационных технологий и методов объектно-ориентированного программирования предоставляет новые возможности для создания эффективных систем электронного документооборота, адаптированных под специфические потребности строительной отрасли. Особую важность в этом контексте приобретает разработка специализированного программного обеспечения, способного не только хранить документы, но и обеспечивать их быстрый поиск, категоризацию, контроль доступа и управление жизненным циклом.

Данная курсовая работа посвящена проектированию и разработке прикладного программного продукта «Строительство. Хранилище проектной документации». Основная цель работы заключается в создании современной системы управления строительной документацией, основанной на принципах

объектно-ориентированного программирования и реализующей эффективные алгоритмы работы с данными.

Актуальность разработки обусловлена следующими потребностями строительной отрасли:

1. Централизация хранения – создание единого репозитория для всех проектных документов

2. Оптимизация поиска – внедрение современных алгоритмов для быстрого нахождения документов

3. Повышение доступности – обеспечение одновременного доступа для нескольких специалистов

4. Улучшение управляемости – внедрение системы контроля версий и отслеживания изменений

5. Снижение затрат – уменьшение расходов на физическое хранение и обслуживание архивов

В рамках выполнения работы были решены следующие основные задачи:

- Проведен анализ предметной области и потребностей пользователей

- Разработана объектно-ориентированная архитектура системы

- Реализованы ключевые классы для работы с документами и их метаданными

- Внедрен эффективный алгоритм триграммного поиска для быстрого нахождения документов

- Создан интуитивно понятный графический интерфейс на основе Windows API

- Обеспечена надежность хранения данных и защита от потери информации

- Проведено комплексное тестирование функциональности системы

Особое внимание в работе уделялось применению современных методологий объектно-ориентированного проектирования, принципам SOLID, паттернам проектирования и лучшим практикам разработки на языке C++. Разработанная система демонстрирует практическое применение теоретических знаний, полученных в ходе изучения дисциплины «Программирование», и служит примером создания полноценного прикладного решения для реальной предметной области.

Значимость работы заключается в том, что разработанный программный продукт может быть использован в реальных строительных организациях для повышения эффективности управления проектной документацией. Система способствует сокращению времени на поиск документов, уменьшению количества ошибок при работе с документацией, повышению контроля за доступом к информации и оптимизации процессов документооборота.

Результатом работы является готовый к использованию программный продукт, который сочетает в себе современные технологии обработки данных, удобный пользовательский интерфейс и надежную архитектуру, обеспечивающую стабильную работу в условиях реальной эксплуатации в строительных организациях различного масштаба.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Проблематика и актуальность

В современной строительной отрасли управление проектной документацией представляет собой сложный и многоаспектный процесс. Строительные проекты сопровождаются большим объемом документации, включающей:

- Чертежи и схемы
- Технические задания
- Сметы и калькуляции
- Разрешительные документы
- Акты выполненных работ
- Протоколы согласований

Традиционные методы хранения на бумажных носителях имеют ряд существенных недостатков:

1. **Физическая уязвимость** – документы могут быть повреждены, утеряны или испорчены.
2. **Ограниченный доступ** – одновременная работа с документами несколькими специалистами затруднена.
3. **Неэффективный поиск** – нахождение нужного документа требует значительных временных затрат.
4. **Отсутствие версионности** – сложно отслеживать изменения в документах.

5. **Высокие затраты на хранение** – требуются специальные помещения и оборудование.

В условиях цифровой трансформации строительной отрасли актуальной задачей является создание специализированного программного обеспечения для электронного хранения и управления проектной документацией.

1.2 Анализ существующих решений

На рынке представлены различные системы управления документами:

1. **Системы электронного документооборота (СЭД)**:
    - "1С:Документооборот"
    - "Directum"
    - "Docsvision"
    - *Преимущества*: комплексный функционал, интеграция с другими системами.
    - *Недостатки*: высокая стоимость, избыточность функционала, сложность внедрения.
2. **Специализированные CAD-системы**:
    - AutoCAD
    - ArchiCAD
    - Revit
    - *Преимущества*: работа с чертежами, 3D-моделирование.
    - *Недостатки*: ориентированы на проектирование, а не на хранение документов.
3. **Облачные хранилища**:
    - Google Drive
    - Dropbox
    - Яндекс.Диск
    - *Преимущества*: доступность, простота использования.
    - *Недостатки*: отсутствие специализированных функций для строительной документации.

1.3 Цель и задачи проекта

Цель проекта: разработка специализированного программного обеспечения для хранения, каталогизации и поиска проектной документации в строительных организациях.

Задачи проекта:

1. Анализ предметной области и требований пользователей.
2. Проектирование архитектуры программного обеспечения.
3. Разработка объектно-ориентированной модели системы.
4. Реализация графического интерфейса на основе Windows API.
5. Внедрение эффективного алгоритма поиска документов.
6. Обеспечение надежного хранения данных.
7. Тестирование и отладка программного продукта.
8. Разработка документации по эксплуатации.

2 ТЕХНИЧЕСКОЕ ЗАДАНИЕ

2.1 Назначение разработки

Разработать прикладной программный продукт «Хранилище проектной документации» для автоматизации процессов учета, хранения и поиска строительной документации.

Область применения: строительные организации, проектные институты, архитектурные бюро.

Классификация по ГОСТ 34.003-90: информационная система.

2.2 Функциональные требования

2.2.1 Основные функции

1. **Управление документами**:
    - Добавление новых документов
    - Редактирование существующих документов
    - Удаление документов
    - Просмотр списка документов
2. **Поиск документов**:
    - Полнотекстовый поиск по всем полям

- Фильтрация по критериям
- Использование алгоритма триграммного поиска
- Поддержка нечеткого поиска

3. **Работа с файлами**:
   - Привязка файлов к записям документов
   - Проверка существования файлов
   - Отображение информации о размере файлов
4. **Импорт/экспорт данных**:
   - Сохранение данных в текстовый файл
   - Загрузка данных из файла
   - Поддержка формата UTF-8
   - Автоматическое устранение дубликатов

2.2.2 Дополнительные функции

1. **Валидация данных**:
   - Проверка обязательных полей
   - Валидация форматов данных
   - Проверка существования файлов
2. **Статистика**:
   - Подсчет общего количества документов
   - Отображение количества найденных документов
   - Информация о размере хранилища

2.3 Требования к интерфейсу пользователя

2.3.1 Общие требования

1. **Стандартизация**: соответствие принципам Windows GUI.
2. **Интуитивность**: понятная навигация и организация элементов.
3. **Доступность**: поддержка стандартных клавиатурных комбинаций.
4. **Адаптивность**: корректное отображение при изменении размеров окна.

2.3.2 Структура интерфейса

1. **Главное окно**:
    - Панель инструментов с кнопками управления
    - Таблица для отображения документов
    - Панель поиска
    - Строка состояния
2. **Диалоговые окна**:
    - Окно добавления/редактирования документа
    - Диалог выбора файла
    - Окна сообщений и предупреждений

2.4 Требования к надежности и безопасности

1. **Целостность данных**:
    - Проверка формата файлов при сохранении
    - Автоматическое восстановление при ошибках чтения
    - Резервное копирование данных
2. **Защита от ошибок**:
    - Обработка исключительных ситуаций
    - Валидация пользовательского ввода
    - Защита от некорректных данных

2.5 Требования к эргономике

1. **Эффективность**:
    - Минимальное количество действий для выполнения операций
    - Быстрый доступ к часто используемым функциям
2. **Обучаемость**:
    - Понятный интерфейс для новых пользователей
    - Контекстная помощь
3. **Удовлетворенность пользователя**:
    - Эстетичный дизайн

- Быстрый отклик на действия пользователя

2.6 Стадии и этапы разработки

В соответствии с ГОСТ 34.601-90:

1. **Техническое задание** (выполнено)
2. **Эскизный проект**:
   - Разработка архитектуры системы
   - Проектирование интерфейса пользователя
   - Определение форматов данных
3. **Технический проект**:
   - Детальное проектирование классов
   - Разработка алгоритмов
   - Проектирование базы данных
4. **Рабочий проект**:
   - Написание исходного кода
   - Создание графического интерфейса
   - Реализация функционала
5. **Внедрение**:
   - Тестирование системы
   - Разработка документации
   - Обучение пользователей

3 МОДЕЛИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММЫ НА ОСНОВЕ ПРИНЦИПОВ ООП

3.1 Архитектура программного обеспечения

Программа построена по классической трехслойной архитектуре:

1. **Слой представления (Presentation Layer)**:
   - Графический интерфейс на основе Windows API

    - Обработка пользовательского ввода
    - Отображение данных
2. **Бизнес-логика (Business Logic Layer)**:
    - Классы Document и DocumentStorage
    - Алгоритмы поиска и обработки данных
    - Правила валидации
3. **Слой данных (Data Access Layer)**:
    - Работа с файловой системой
    - Сериализация и десериализация данных
    - Управление файлами документов

3.2 Проектирование классов

3.2.1 Класс Document

Назначение: представление документа с метаданными.

Атрибуты:

- name: название документа (wstring)
- author: автор документа (wstring)
- date: дата создания (wstring)
- filePath: путь к файлу (wstring)
- nameTrigrams: триграммы названия для поиска
- authorTrigrams: триграммы автора для поиска

Методы:

- Конструкторы и деструктор
- Геттеры и сеттеры
- isValid(): проверка корректности данных
- toString(): сериализация в строку
- fromString(): десериализация из строки
- updateTrigrams(): обновление триграмм для поиска

Особенности реализации:

- Использование Unicode (wstring)
- Поддержка триграммного поиска
- Встроенная валидация данных

3.2.2 Класс DocumentStorage

Назначение: управление коллекцией документов.

Атрибуты:

- documents: основная коллекция документов (vector<Document>)
- searchResults: результаты поиска (vector<Document>)
- isSearchMode: флаг режима поиска (bool)
- lastSearchQuery: последний поисковый запрос (wstring)

Методы:

- addDocument(): добавление документа
- editDocument(): редактирование документа
- deleteDocument(): удаление документа
- searchDocuments(): поиск документов
- clearSearch(): очистка результатов поиска
- saveToFile(): сохранение в файл
- loadFromFile(): загрузка из файла

Алгоритмы:

- Алгоритм триграммного поиска
- Алгоритм устранения дубликатов
- Алгоритм сериализации/десериализации

3.2.3 Класс GUIWindow (неявный)

Назначение: управление графическим интерфейсом.

Компоненты:

- Оконная процедура WndProc
- Диалоговая процедура DialogProc
- Элементы управления Windows API

3.3 Диаграмма классов

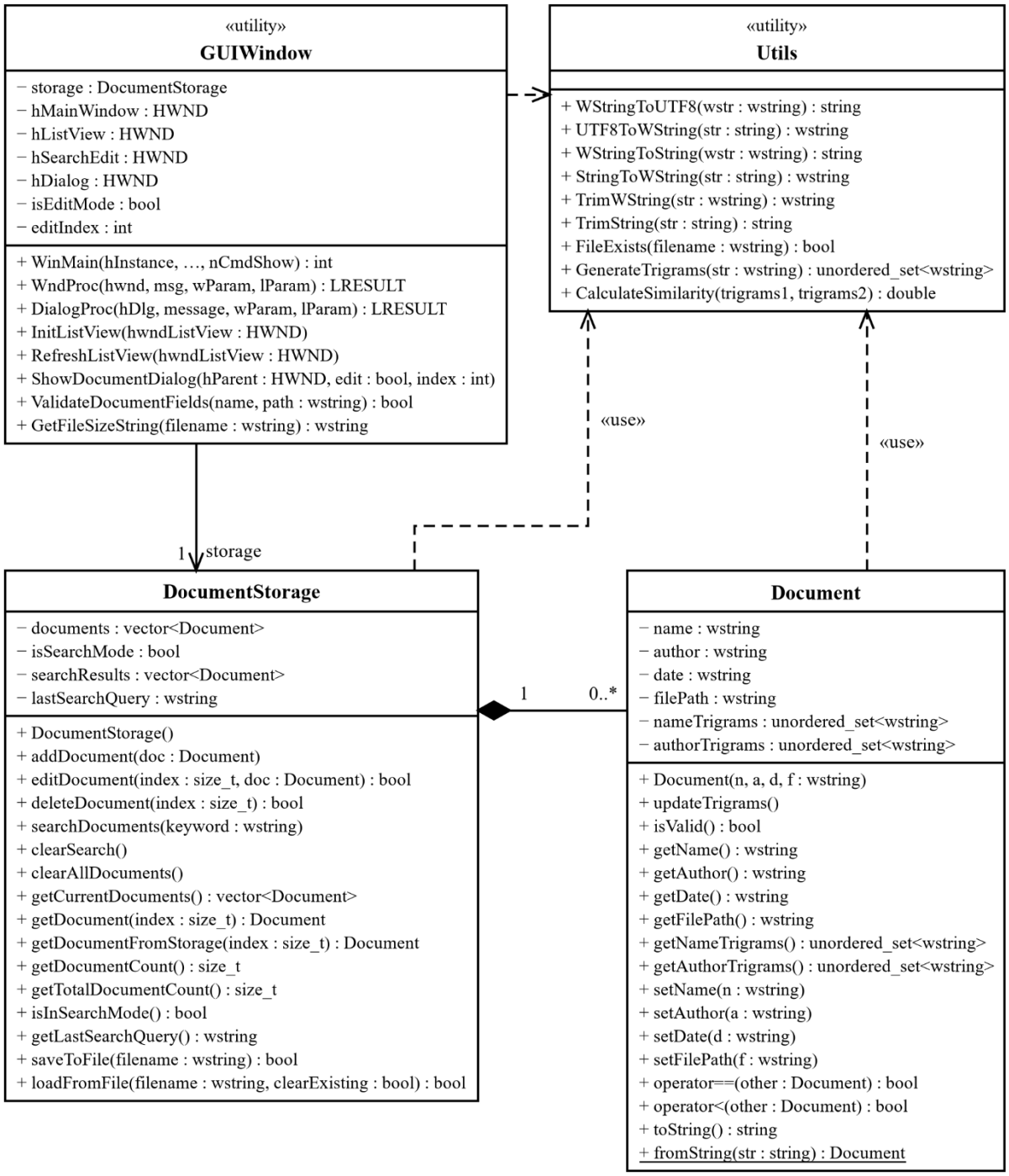


Рисунок 1 – Диаграмма классов

3.4 Алгоритмы и методы

3.4.1 Алгоритм триграммного поиска

Триграмма – последовательность из трех символов. Алгоритм основан на сравнении триграмм поискового запроса и текста документа.

Этапы алгоритма:

1. **Генерация триграмм:**

```
for i from 0 to length(text) - 3:
   trigram = text.substr(i, 3)
   add trigram to set
```

2. **Вычисление меры сходства** (коэффициент Жаккара):

$$similarity = |A \cap B| / |A \cup B|$$

где A – триграммы запроса, B – триграммы текста

3. **Применение порога:**
   - Документ считается релевантным, если similarity ≥ 0.3
   - Для коротких запросов (≤3 символов) используется точное совпадение

Преимущества:

- Устойчивость к опечаткам
- Поддержка нечеткого поиска
- Быстрая обработка

3.4.2 Алгоритм сериализации данных

Формат файла:

- Кодировка: UTF-8 с BOM
- Разделитель полей: точка с запятой (;)
- Разделитель записей: CRLF (\r\n)

Структура записи:

название; автор; дата; путь_к_файлу

Обработка особых случаев:

- Экранирование специальных символов
- Обработка пустых полей
- Проверка целостности данных

3.5 Проектирование базы данных

В данной реализации используется файловое хранение данных вместо СУБД.

Структура хранения:

1. **Основной файл данных** (documents.txt):
    - Текстовый формат с разделителями
    - UTF-8 кодировка
    - Автоматическое резервное копирование
2. **Индексы для быстрого поиска**:
    - Триграммы хранятся в оперативной памяти
    - Автоматическое обновление при изменении данных

Преимущества файлового хранения:

- Простота реализации
- Отсутствие зависимости от СУБД
- Легкость резервного копирования
- Портативность данных

Недостатки:

- Ограниченная производительность при больших объемах данных
- Отсутствие транзакций
- Ограниченные возможности сложных запросов

4 ОПИСАНИЕ ПРОГРАММЫ

4.1 Функциональное назначение

Программа предназначена для автоматизации процессов учета и управления проектной документацией в строительных организациях.

Основные функции:

1. **Каталогизация документов**:
    - Создание структурированного каталога
    - Присвоение метаданных документам
    - Организация по категориям

2. **Быстрый поиск**:
    - Полнотекстовый поиск по всем полям
    - Фильтрация по критериям
    - История поисковых запросов
3. **Управление файлами**:
    - Связь метаданных с физическими файлами
    - Проверка целостности файлов
    - Информация о размере файлов
4. **Импорт/экспорт**:
    - Обмен данными с другими системами
    - Резервное копирование
    - Миграция данных

4.2 Описание логической структуры

4.2.1 Модульная структура

Программа состоит из следующих модулей:

1. **main.cpp** – точка входа, инициализация приложения
2. **document.h/cpp** – класс Document и связанные функции
3. **storage.h/cpp** – класс DocumentStorage
4. **gui.h/cpp** – функции графического интерфейса
5. **utils.h/cpp** – вспомогательные функции

4.2.2 Поток данных

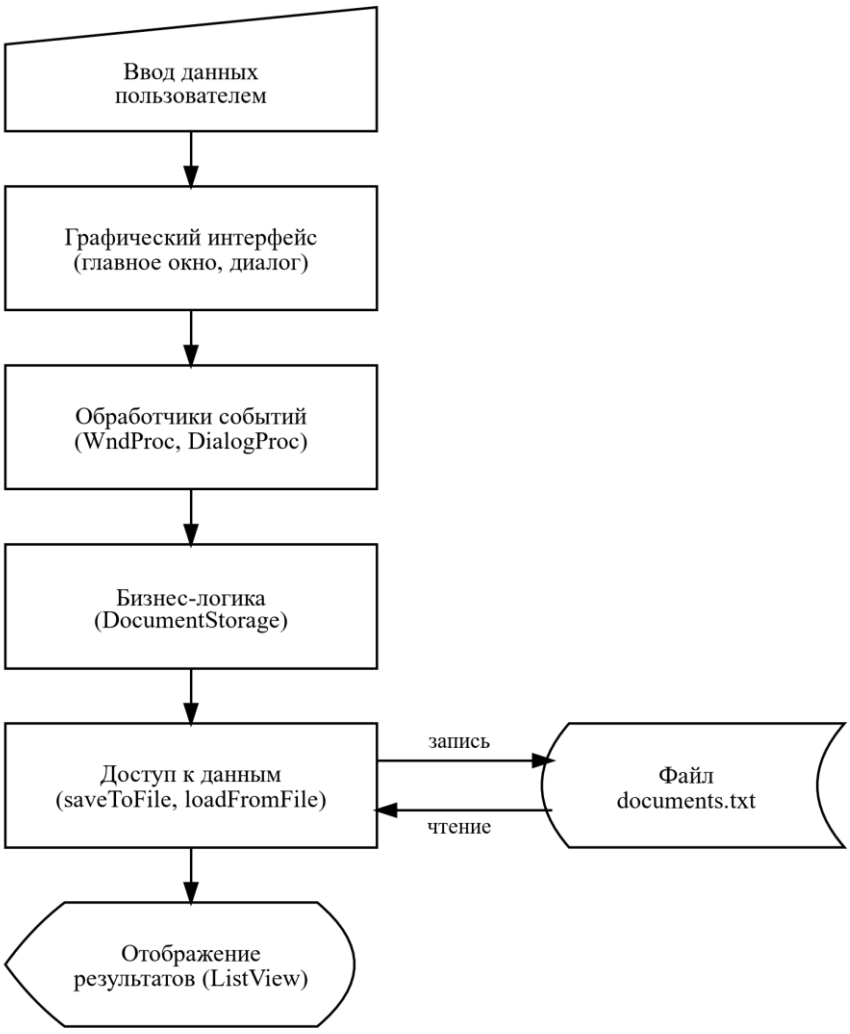


Рисунок 2 – Поток данных

4.2.3 Структура данных в памяти

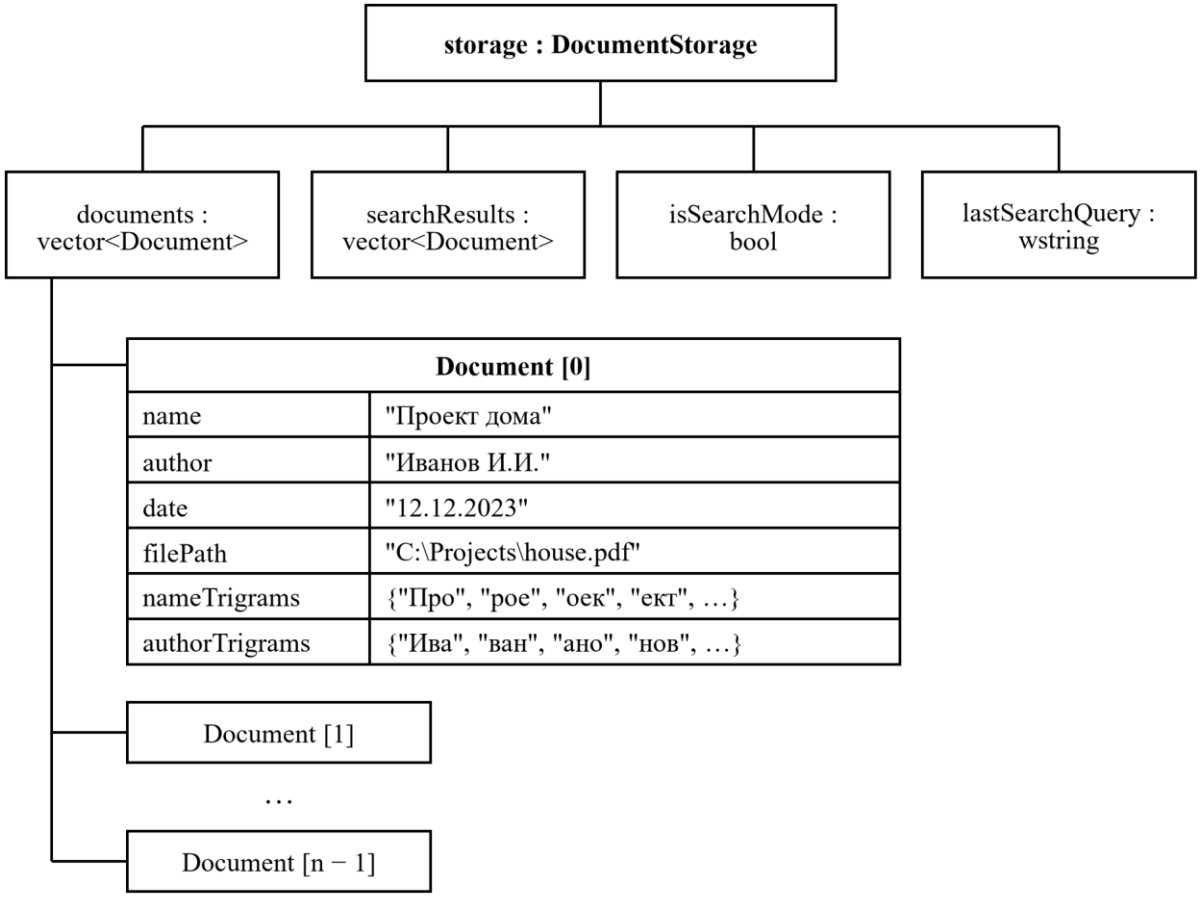


Рисунок 3 – Структура данных в памяти

4.3 Входные данные

4.3.1 Типы входных данных

1. **Метаданные документов**:
   - Название документа (строка, обязательное поле)
   - Автор документа (строка, необязательное поле)
   - Дата создания (строка в формате ДД.ММ.ГГГГ)
   - Путь к файлу (строка, обязательное поле)
2. **Файлы документов**:
   - Любые типы файлов (PDF, DOC, DWG, и т.д.)
   - Локальные пути или сетевые пути
3. **Команды пользователя**:
   - Добавление/редактирование/удаление
   - Поисковые запросы
   - Команды импорта/экспорта

4.3.2 Источники входных данных

1. **Ручной ввод** через графический интерфейс
2. **Импорт из файла** documents.txt
3. **Выбор файлов** через диалог открытия файла

4.4 Выходные данные

4.4.1 Форматы выходных данных

1. **Визуальное представление**:
   - Таблица документов в главном окне
   - Диалоговые окна с результатами операций
   - Сообщения об ошибках и предупреждения
2. **Файловый вывод**:
   - Файл documents.txt с данными в формате UTF-8
   - Логи работы программы (при необходимости)
3. **Статистическая информация**:
   - Количество документов в хранилище
   - Количество найденных документов
   - Информация о размере файлов

4.4.2 Типы сообщений

1. **Информационные сообщения**:
   - Подтверждение успешных операций
   - Статус выполнения операций
   - Справка по использованию
2. **Предупреждения**:
   - Потенциальные проблемы (несуществующие файлы)
   - Рекомендации пользователю
   - Подтверждение опасных операций
3. **Сообщения об ошибках**:
   - Ошибки ввода данных
   - Ошибки доступа к файлам
   - Системные ошибки

4.5 Описание модулей программы

4.5.1 Модуль Document

Назначение: представление и обработка данных документа.

Основные функции:

1. **Инициализация и валидация**:

```
Document doc(name, author, date, filePath);
if (doc.isValid()) {
    // Обработка документа
}
```

2. **Сериализация**:

```
string serialized = doc.toString();
Document restored = Document::fromString(serialized);
```

3. **Поисковые функции**:

```
auto trigrams = doc.getNameTrigrams();
double similarity = CalculateSimilarity(queryTrigrams, trigrams)
```

4.5.2 Модуль DocumentStorage

Назначение: управление коллекцией документов.

Основные функции:

1. **CRUD операции**:

```
storage.addDocument(doc);
storage.editDocument(index, newDoc);
storage.deleteDocument(index);
```

2. **Поиск**:

```
storage.searchDocuments(L"проект дома");
auto results = storage.getCurrentDocuments();
```

3. **Сохранение/загрузка**:

```
storage.saveToFile(L"documents.txt");
storage.loadFromFile(L"documents.txt");
```

4.5.3 Модуль графического интерфейса

Назначение: взаимодействие с пользователем.

Основные компоненты:

1. **Главное окно** (WndProc):

- Обработка сообщений Windows
- Управление элементами интерфейса
- Координация работы модулей

2. **Диалоговые окна** (DialogProc):

- Ввод данных документа
- Выбор файлов
- Отображение сообщений

3. **Элементы управления**:

- ListView для отображения таблицы
- Edit controls для ввода текста
- Buttons для управления
- Static controls для меток

4.6 Используемые технологии и библиотеки

4.6.1 Язык программирования и стандарт

- **Язык**: C++17
- **Компилятор: GCC 14.2 (MinGW-w64), среда разработки Code::Blocks 25.03**
- **Стандартная библиотека**: STL

4.6.2 Платформенные API

1. **Windows API**:

- User32.dll – создание окон и элементов управления
- Comctl32.dll – общие элементы управления
- Shell32.dll – работа с файловой системой

2. **Используемые функции**:

```
CreateWindowExW()    // Создание окон
SendMessageW()       // Отправка сообщений
GetOpenFileNameW()   // Диалог выбора файла
MessageBoxW()        // Отображение сообщений
```

4.6.3 Внешние зависимости

1. **Заголовочные файлы Windows**:

```
#include <windows.h>
#include <commctrl.h>
```

2. **Стандартная библиотека C++**:

```
#include <string>
#include <vector>
#include <fstream>
```

```
#include <algorithm>
#include <unordered_set>
```

4.6.4 Кодировки и локализация

- **Внутреннее представление**: UTF-16 (wstring)
- **Файловый ввод/вывод**: UTF-8
- **Графический интерфейс**: Unicode (W-функции)

Функции преобразования:

```
std::string WStringToUTF8(const std::wstring& wstr);
std::wstring UTF8ToWString(const std::string& str);
```

5 ИНСТРУКЦИЯ ПО ЭКСПЛУАТАЦИИ ПРОГРАММЫ

5.1 Назначение программы

Программа «Хранилище проектной документации» предназначена для:

- Учета проектной документации в строительных организациях
- Быстрого поиска документов по различным критериям
- Организации электронного архива документов
- Управления метаданными документов

5.2 Системные требования

5.2.1 Минимальные требования

- **Операционная система**: Windows 7 SP1 или выше
- **Процессор**: 1 GHz или выше
- **Оперативная память**: 512 MB
- **Свободное место на диске**: 50 MB
- **Разрешение экрана**: 1024×768

5.2.2 Рекомендуемые требования

- **Операционная система**: Windows 10/11
- **Процессор**: 2 GHz, 2 ядра
- **Оперативная память**: 2 GB
- **Свободное место на диске**: 100 MB
- **Разрешение экрана**: 1920×1080

5.2.3 Зависимости

- **Библиотеки времени выполнения: не требуются (статическая компоновка MinGW-w64)**
- **Файловая система**: NTFS или FAT32

- **Права доступа**: права на чтение/запись файлов

5.3 Установка и запуск

5.3.1 Установка программы

1. **Распаковка архива**:
   - Создать папку для программы (например, C:\DocStorage)
   - Скопировать файлы программы:
     - DocStorage.exe – исполняемый файл
     - documents.txt – файл данных (создается автоматически)
2. **Проверка зависимостей**:
   - Дополнительные библиотеки не требуются: программа собрана статически
   - Программа запускается на любом компьютере с Windows 7 и выше
3. **Создание ярлыка**:
   - Правой кнопкой мыши на DocStorage.exe
   - Выбрать "Создать ярлык"
   - Перенести ярлык на рабочий стол или в меню "Пуск"

5.3.2 Запуск программы

Способы запуска:

1. **Двойной щелчок** по исполняемому файлу
2. **Запуск из командной строки**:

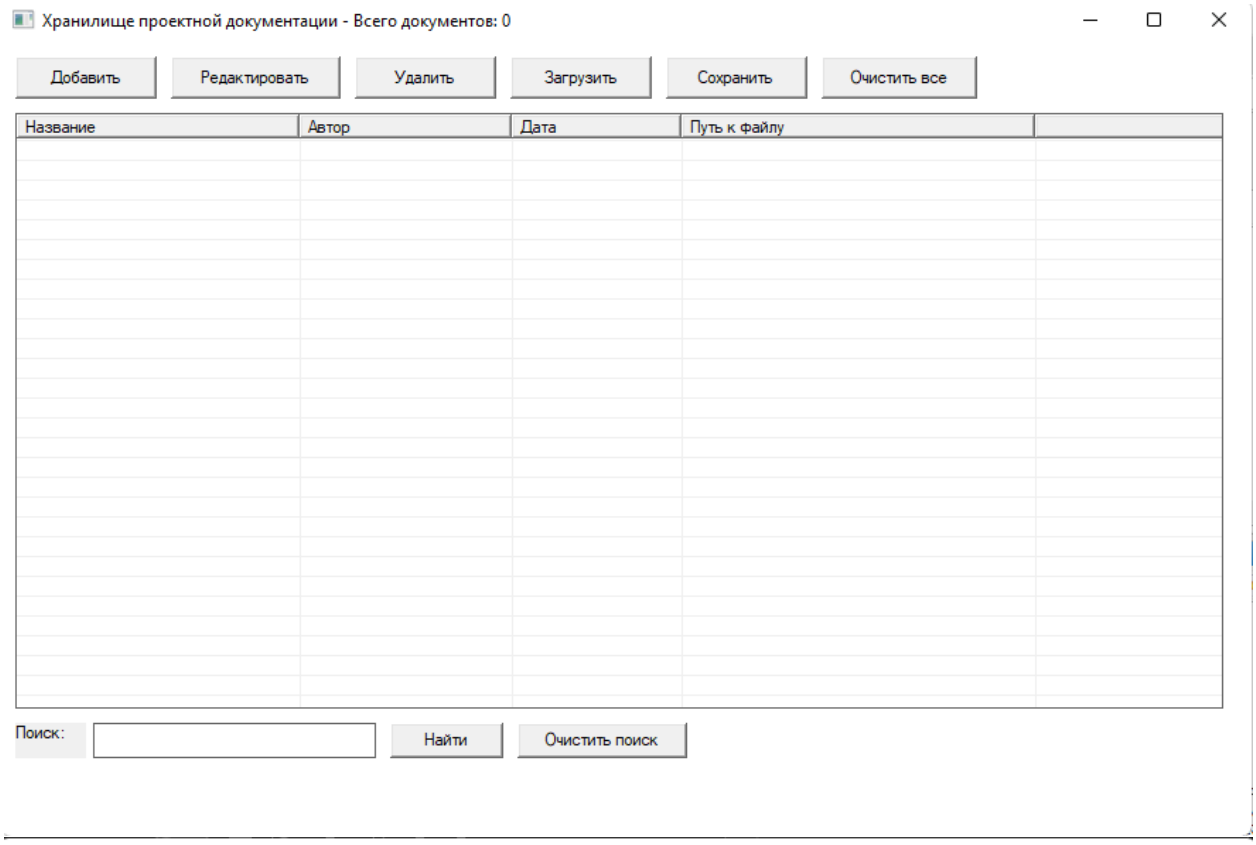
   DocStorage.exe

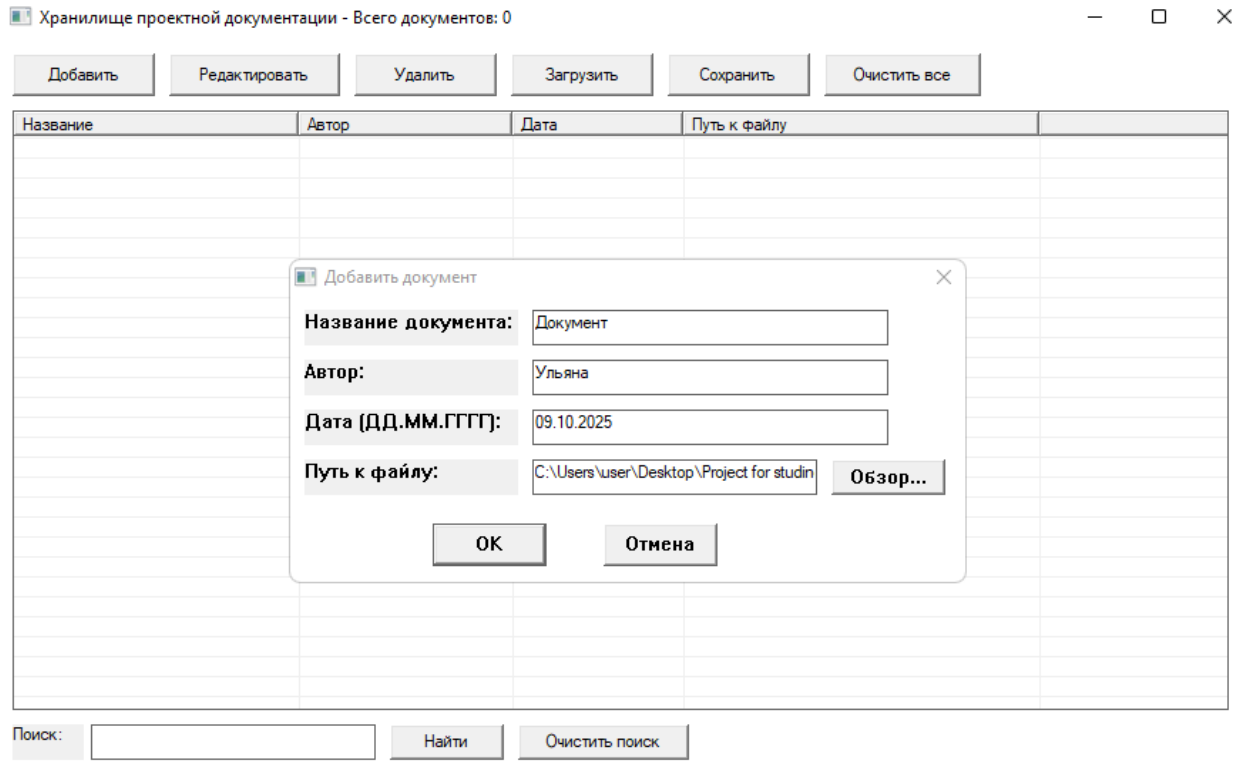
3. **Ярлык на рабочем столе**

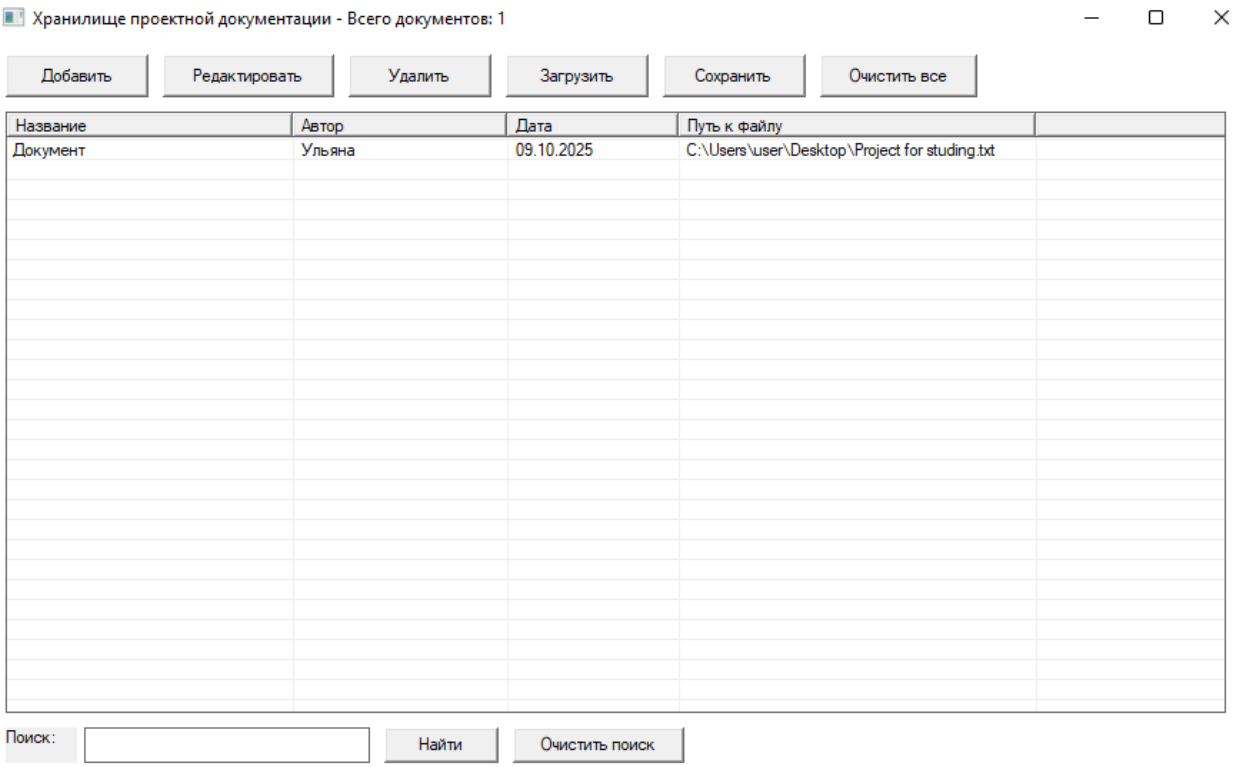
Первый запуск:

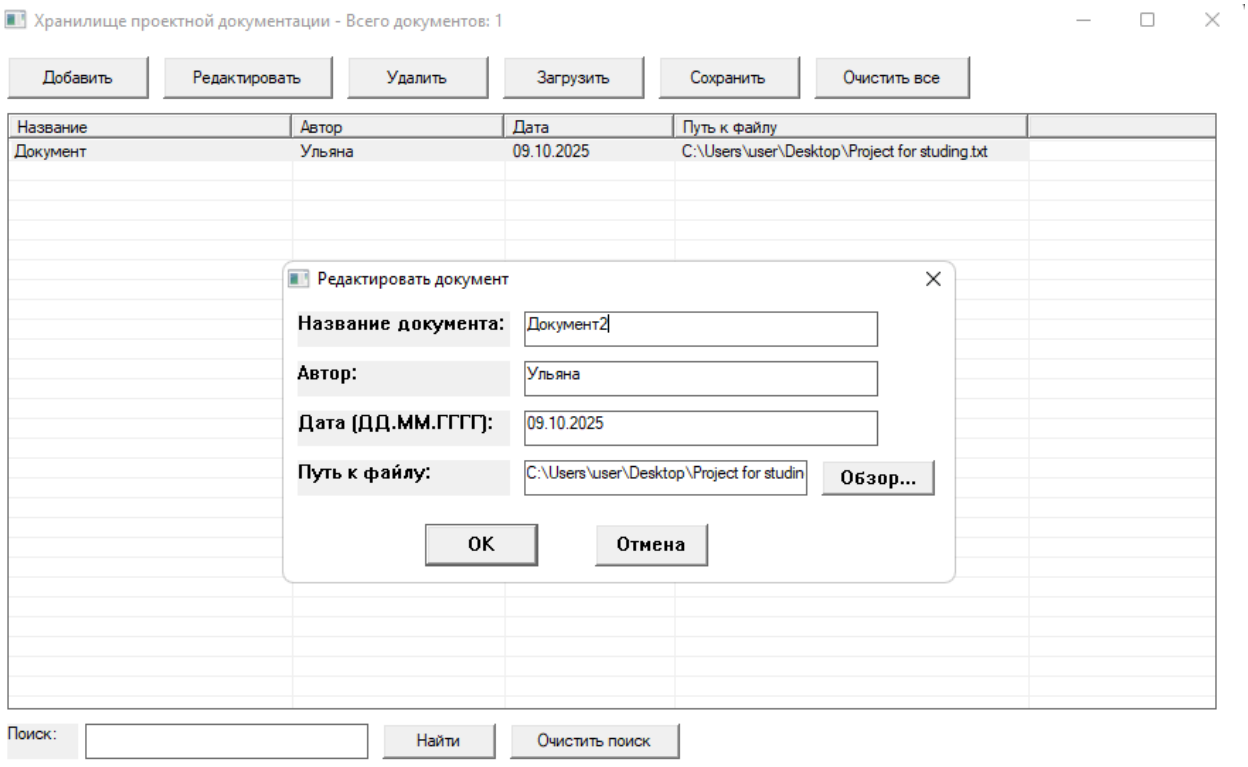
- Создается файл documents.txt в папке с программой
- Инициализируется пустая база данных
- Отображается главное окно программы

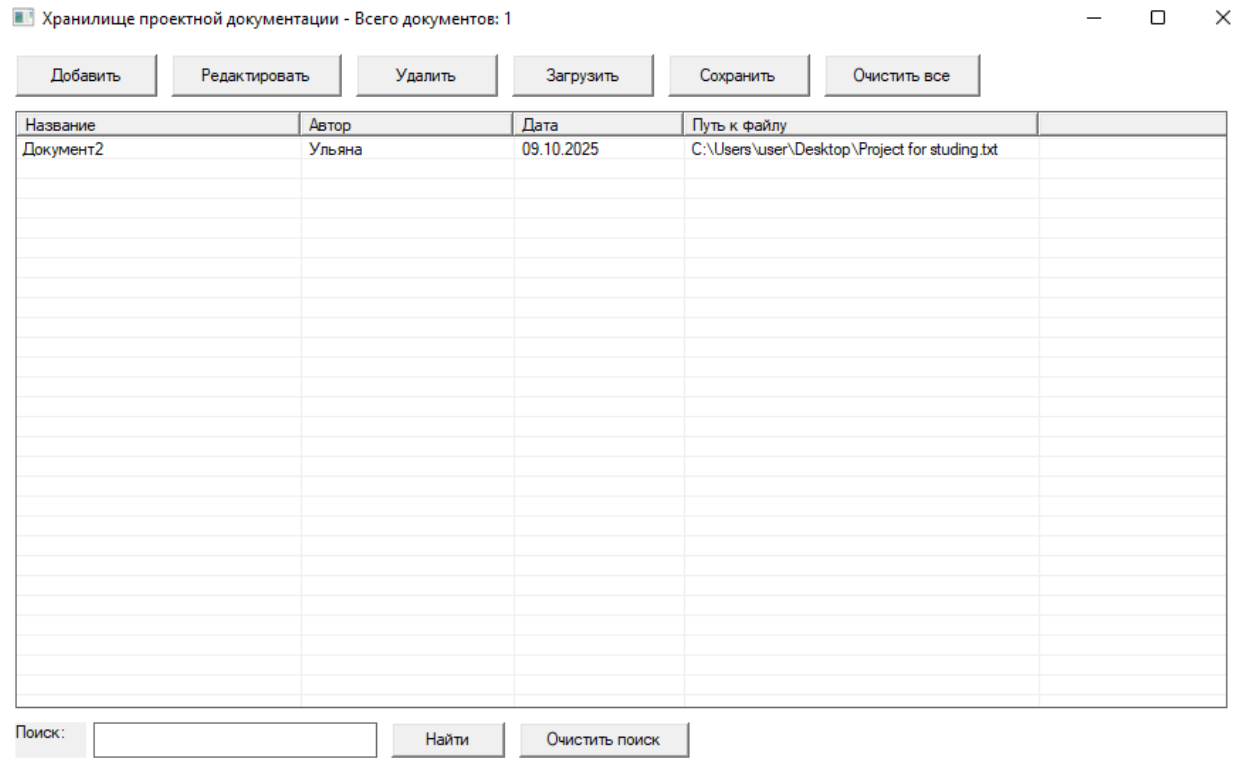
5.4. Интерфейс пользователя.

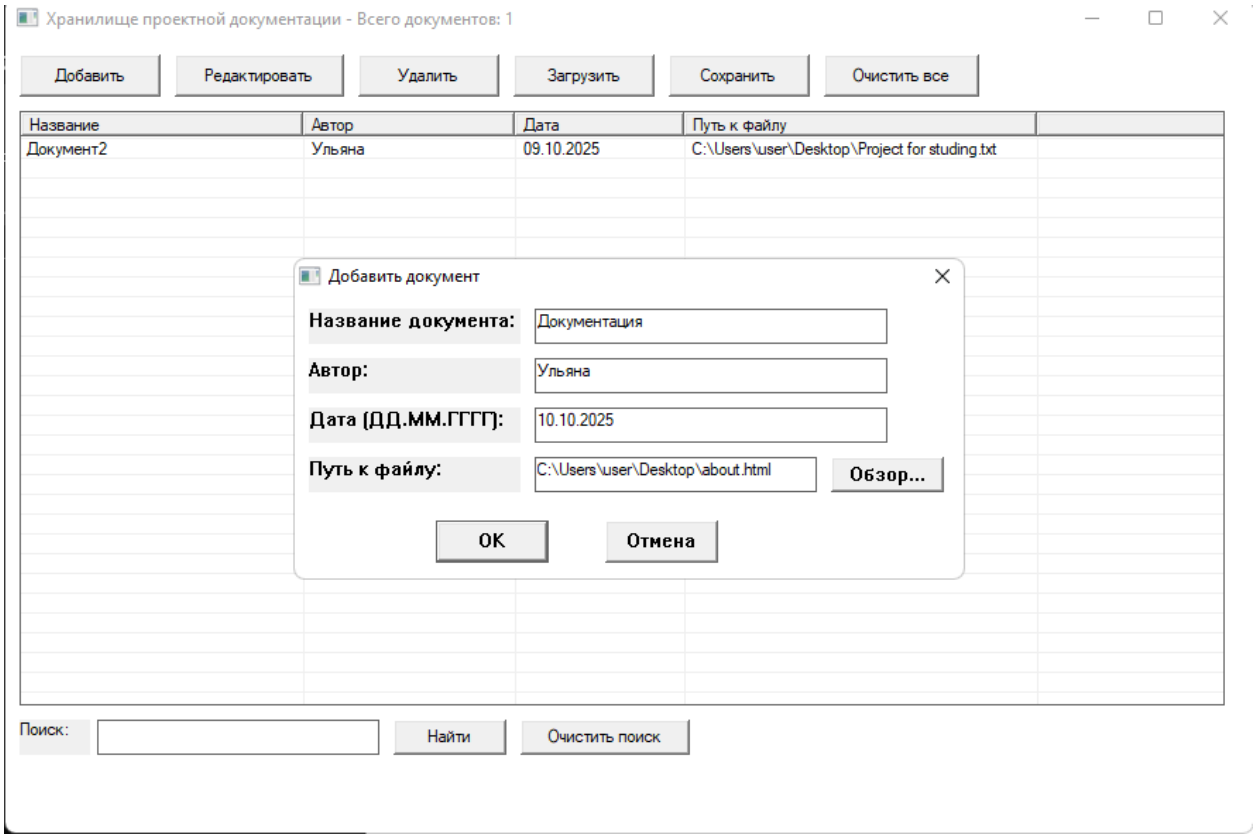


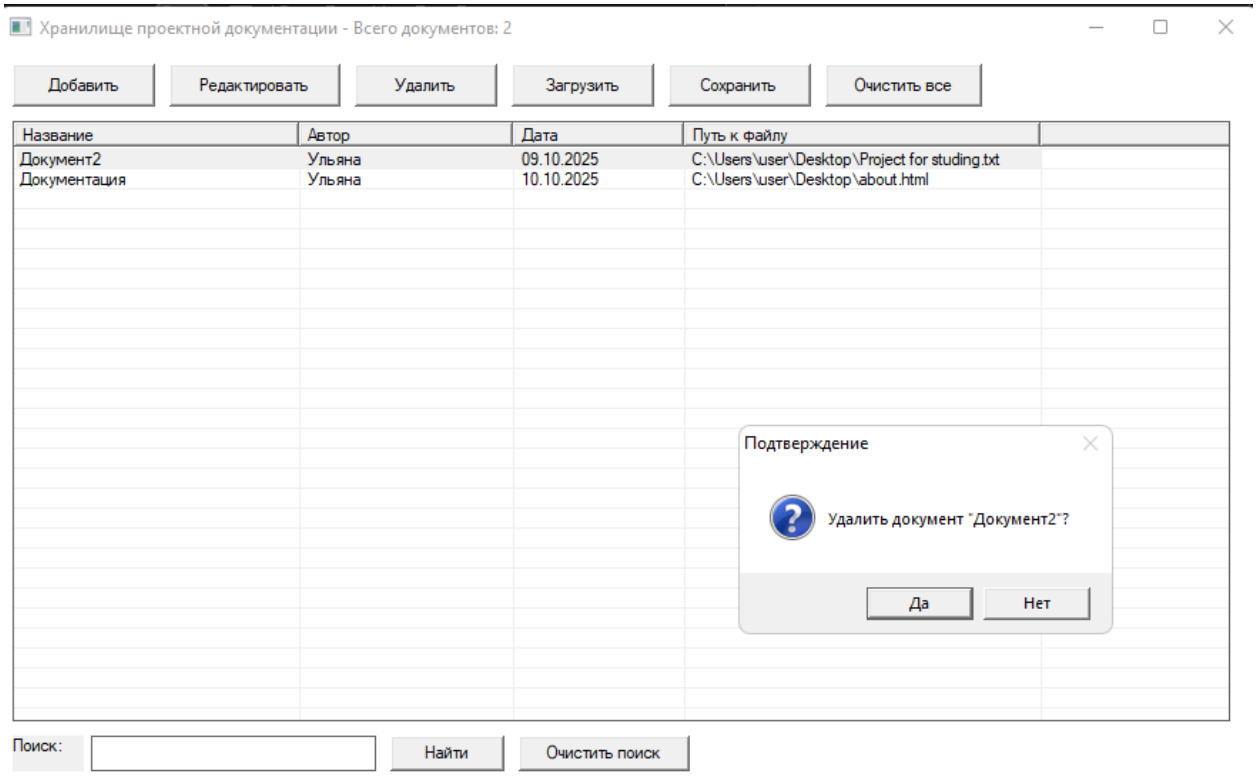


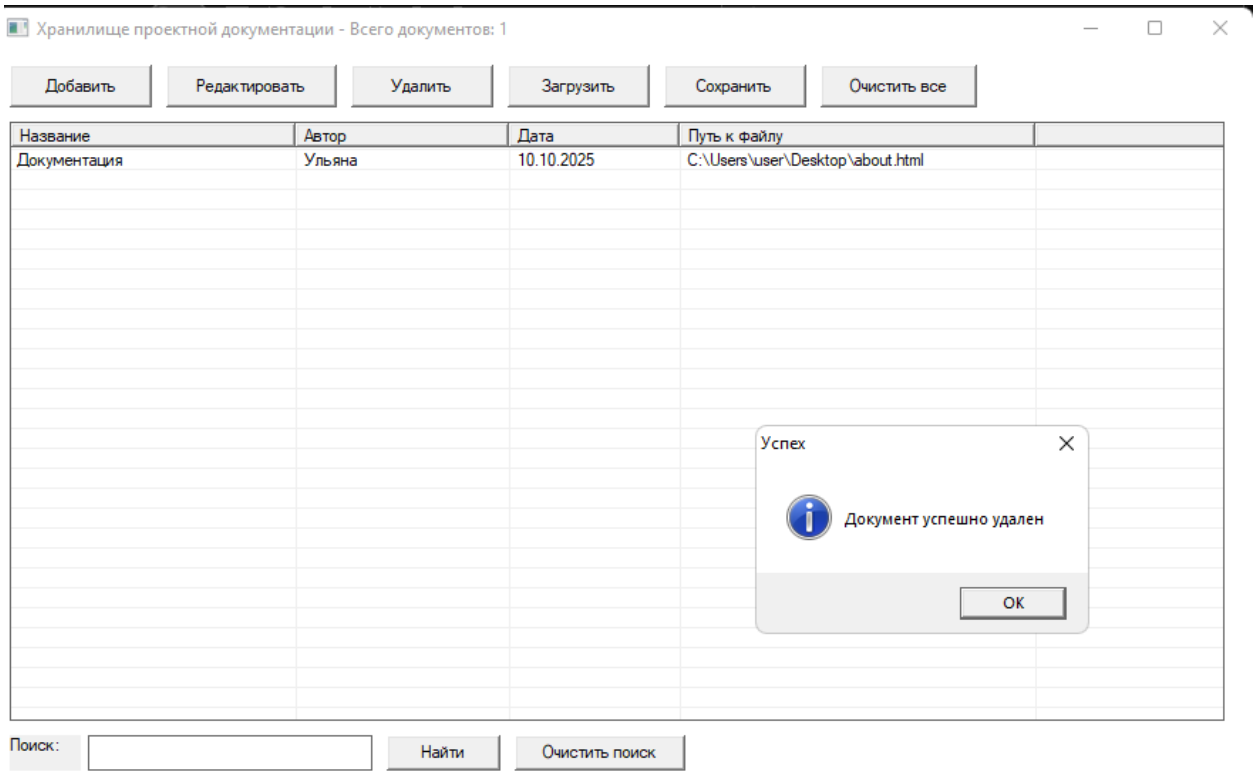

Хранилище проектной документации - Всего документов: 1
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документ
Ульяна
09.10.2025
C:\Users\user\Desktop\Project for studing.txt
Поиск:
Найти
Очистить поиск

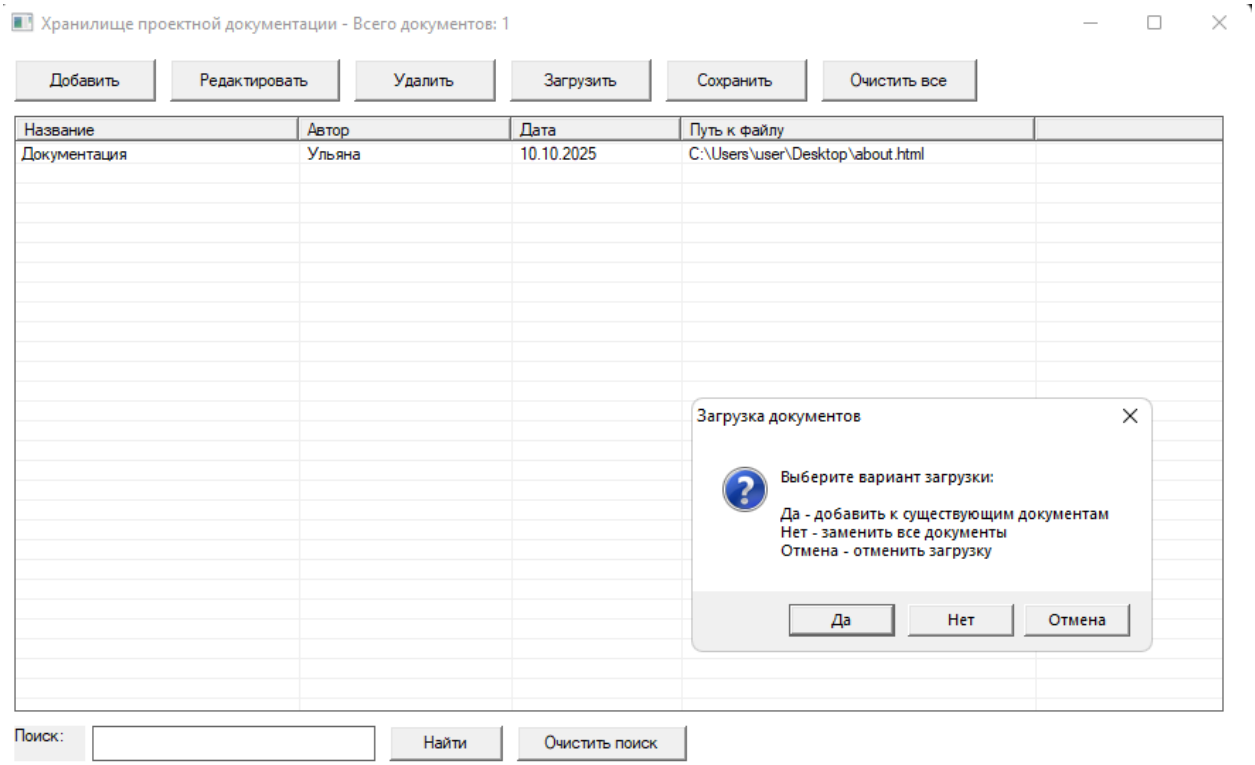

Хранилище проектной документации - Всего документов: 1
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документ
Ульяна
09.10.2025
C:\Users\user\Desktop\Project for studing.txt
Редактировать документ
Название документа:
Документ2
Автор:
Ульяна
Дата (ДД.ММ.ГГГГ):
09.10.2025
Путь к файлу:
C:\Users\user\Desktop\Project for studin
Обзор...
OK
Отмена
Поиск:
Найти
Очистить поиск

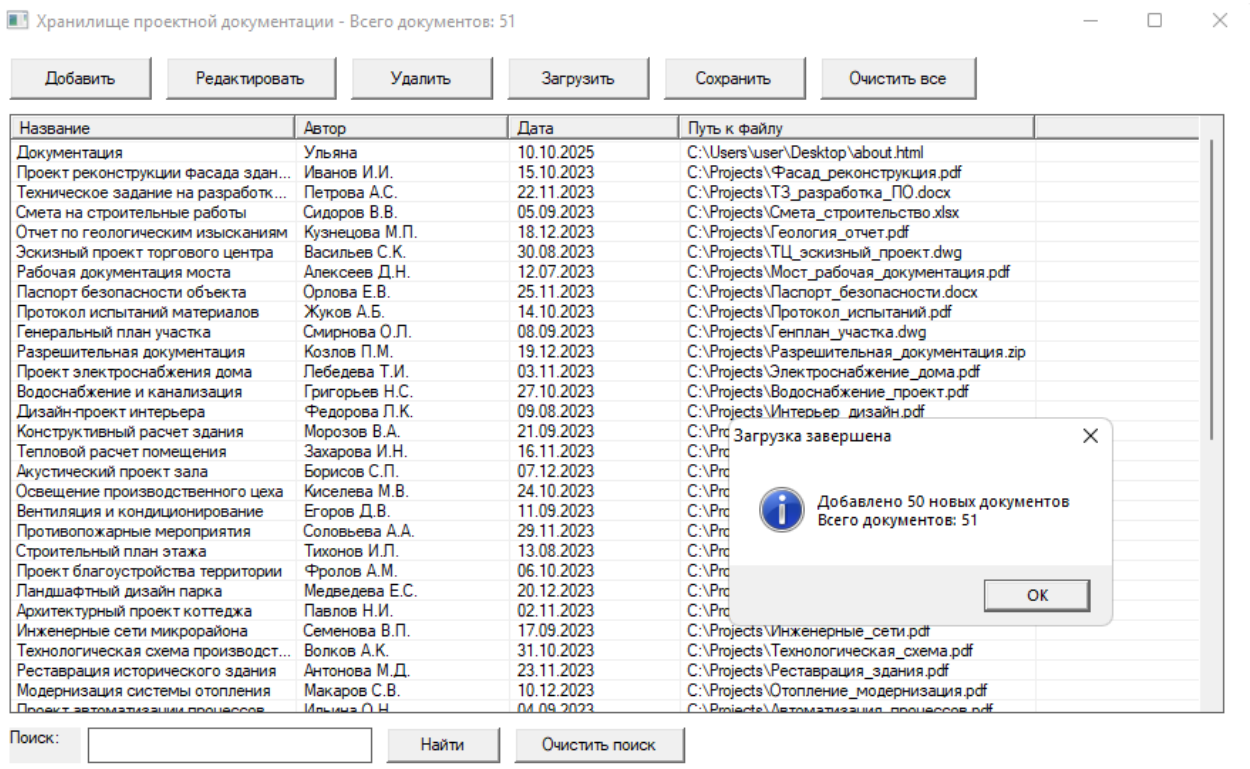

Хранилище проектной документации - Всего документов: 1
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документ2
Ульяна
09.10.2025
C:\Users\user\Desktop\Project for studing.txt
Поиск:
Найти
Очистить поиск

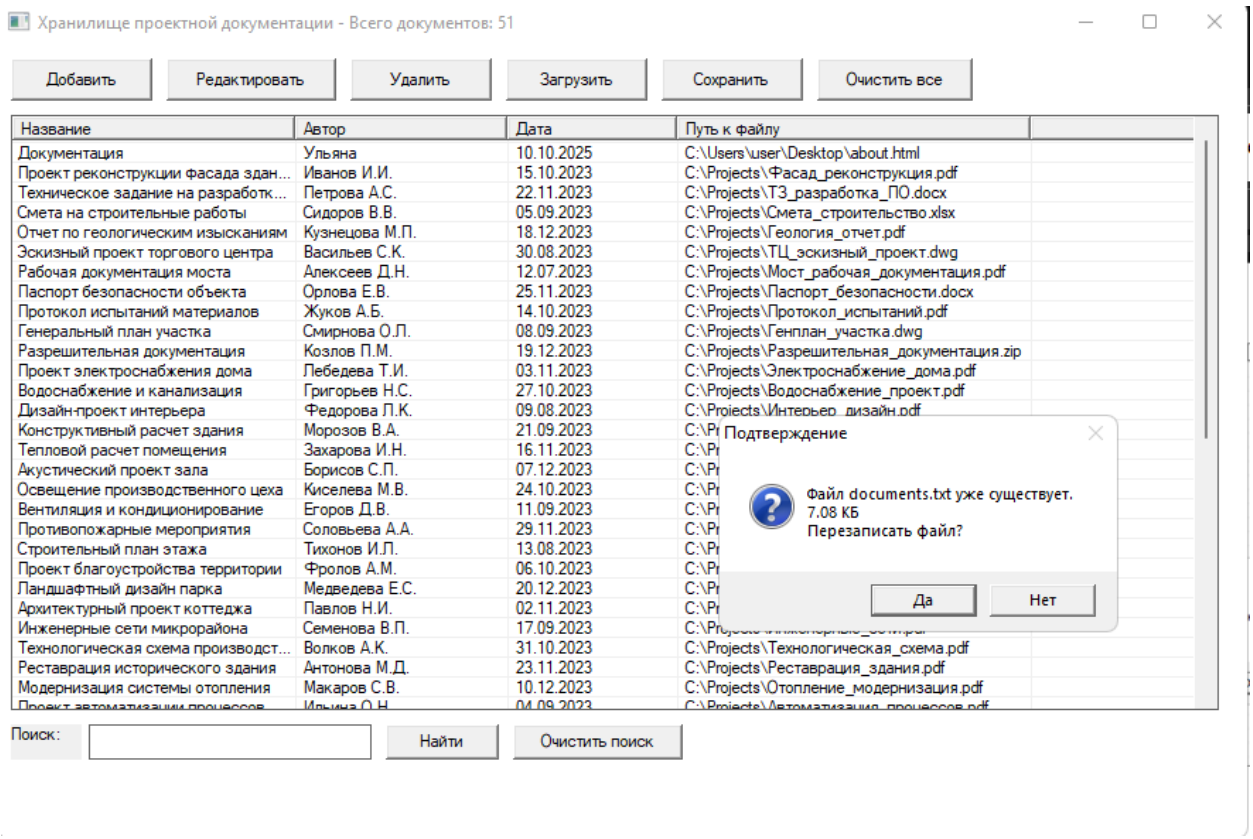

Хранилище проектной документации - Всего документов: 1
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документ2
Ульяна
09.10.2025
C:\Users\user\Desktop\Project for studing.txt
Добавить документ
Название документа:
Документация
Автор:
Ульяна
Дата (ДД.ММ.ГГГГ):
10.10.2025
Путь к файлу:
C:\Users\user\Desktop\about.html
Обзор...
OK
Отмена
Поиск:
Найти
Очистить поиск

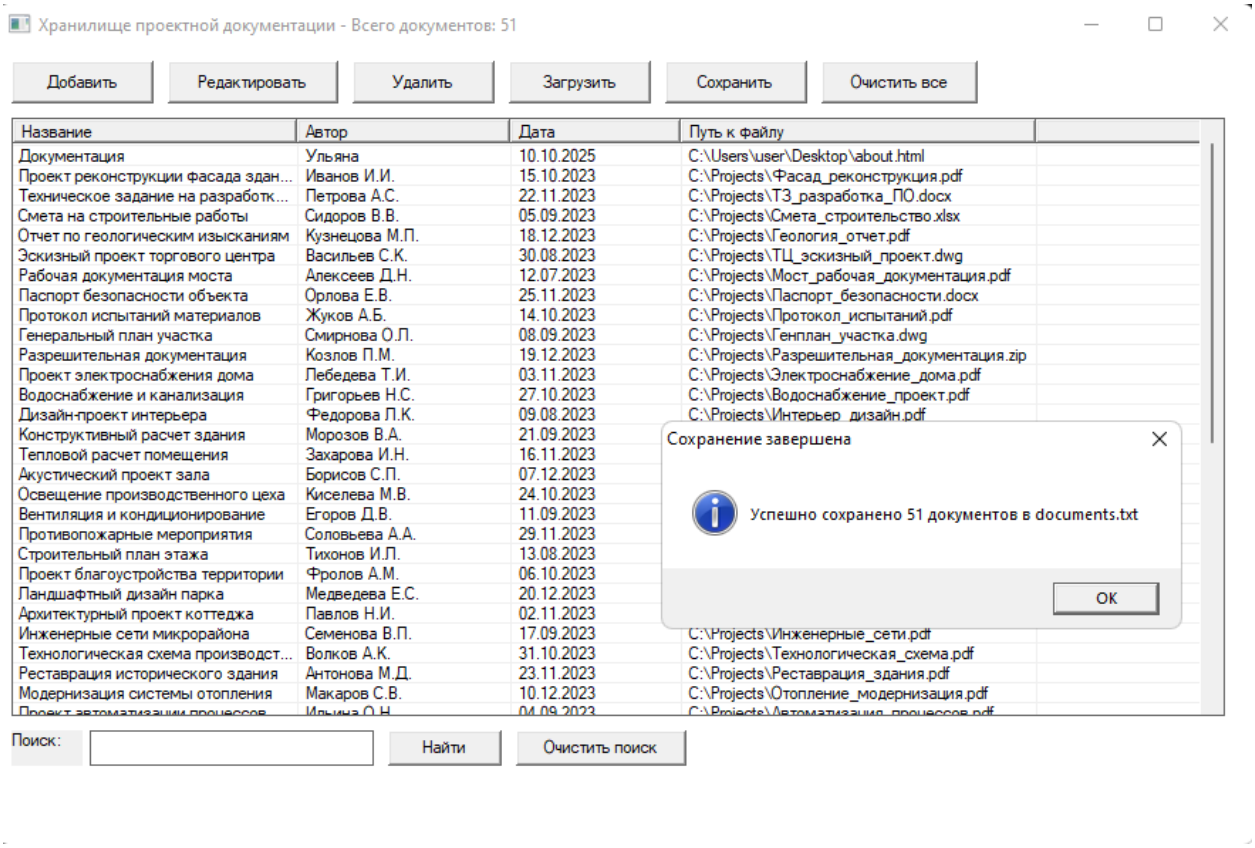

Хранилище проектной документации - Всего документов: 2
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документ2
Ульяна
09.10.2025
C:\Users\user\Desktop\Project for studing.txt
Документация
Ульяна
10.10.2025
C:\Users\user\Desktop\about.html
Подтверждение
Удалить документ "Документ2"?
Да
Нет
Поиск:
Найти
Очистить поиск

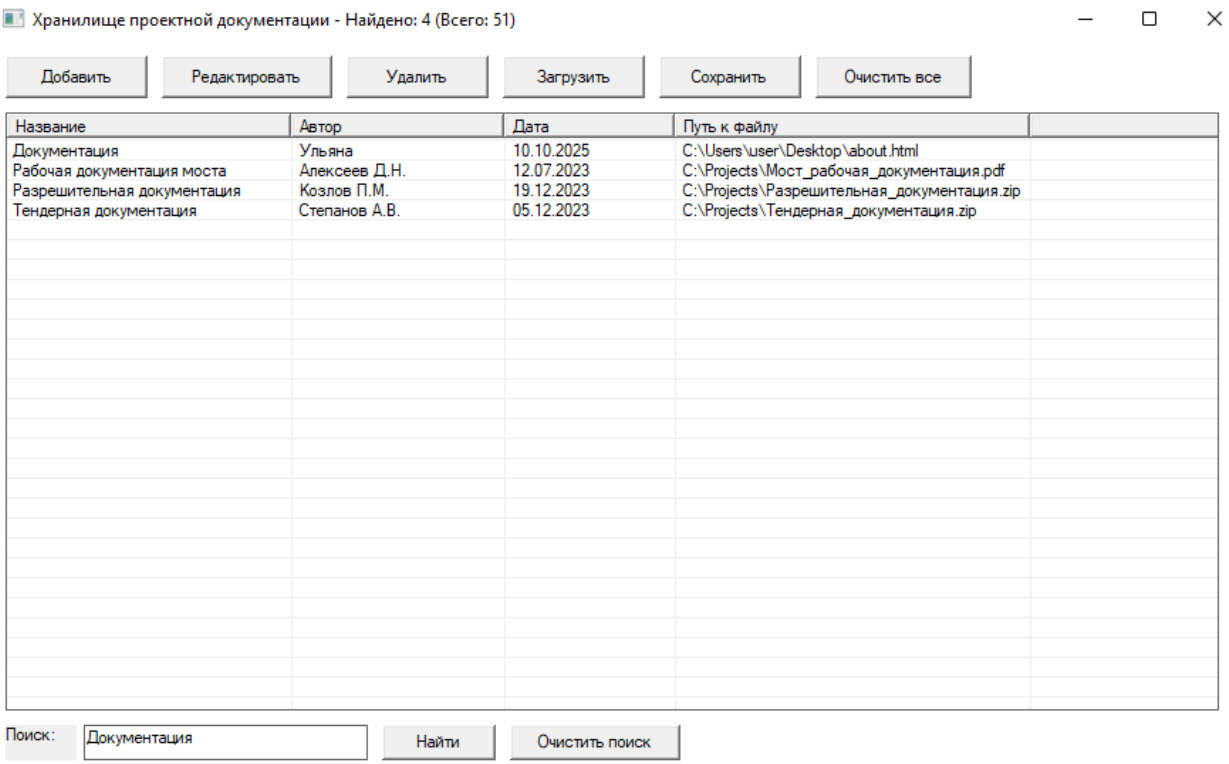

Хранилище проектной документации - Всего документов: 1
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документация
Ульяна
10.10.2025
C:\Users\user\Desktop\about.html
Успех
Документ успешно удален
OK
Поиск:
Найти
Очистить поиск

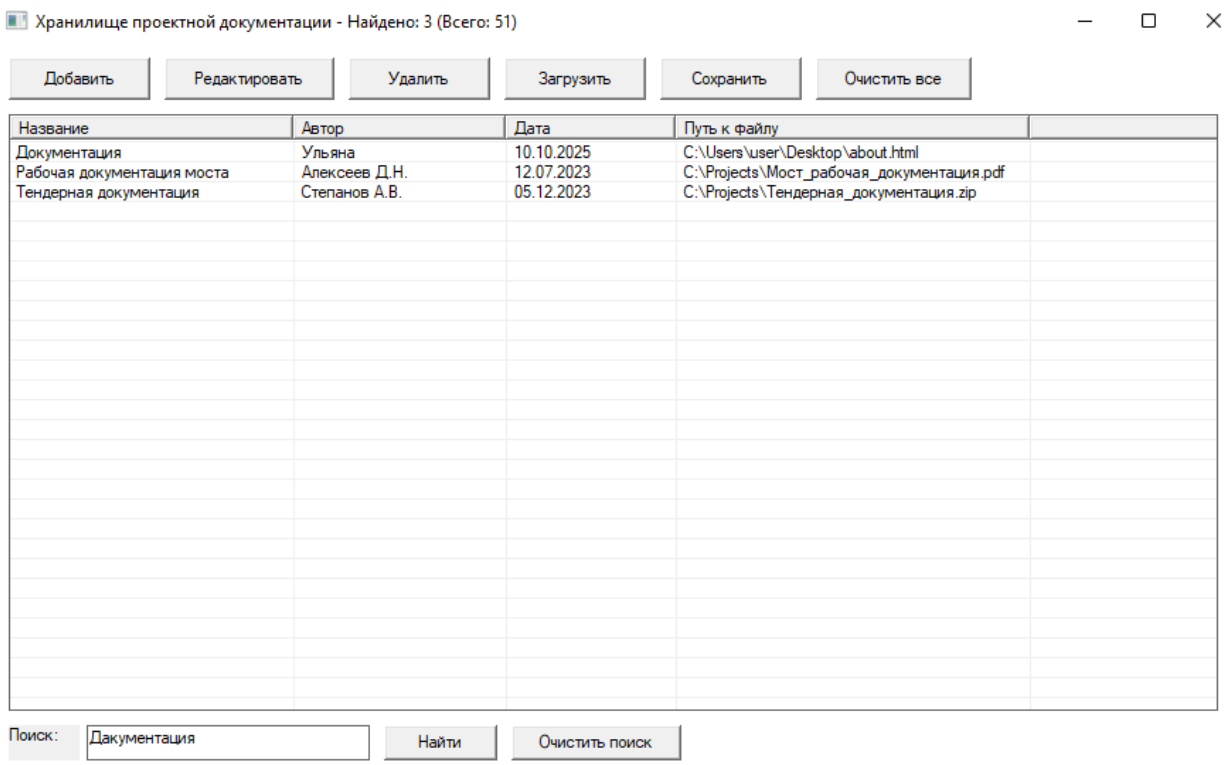

Хранилище проектной документации - Всего документов: 1
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документация
Ульяна
10.10.2025
C:\Users\user\Desktop\about.html
Загрузка документов
Выберите вариант загрузки:
Да - добавить к существующим документам
Нет - заменить все документы
Отмена - отменить загрузку
Да
Нет
Отмена
Поиск:
Найти
Очистить поиск

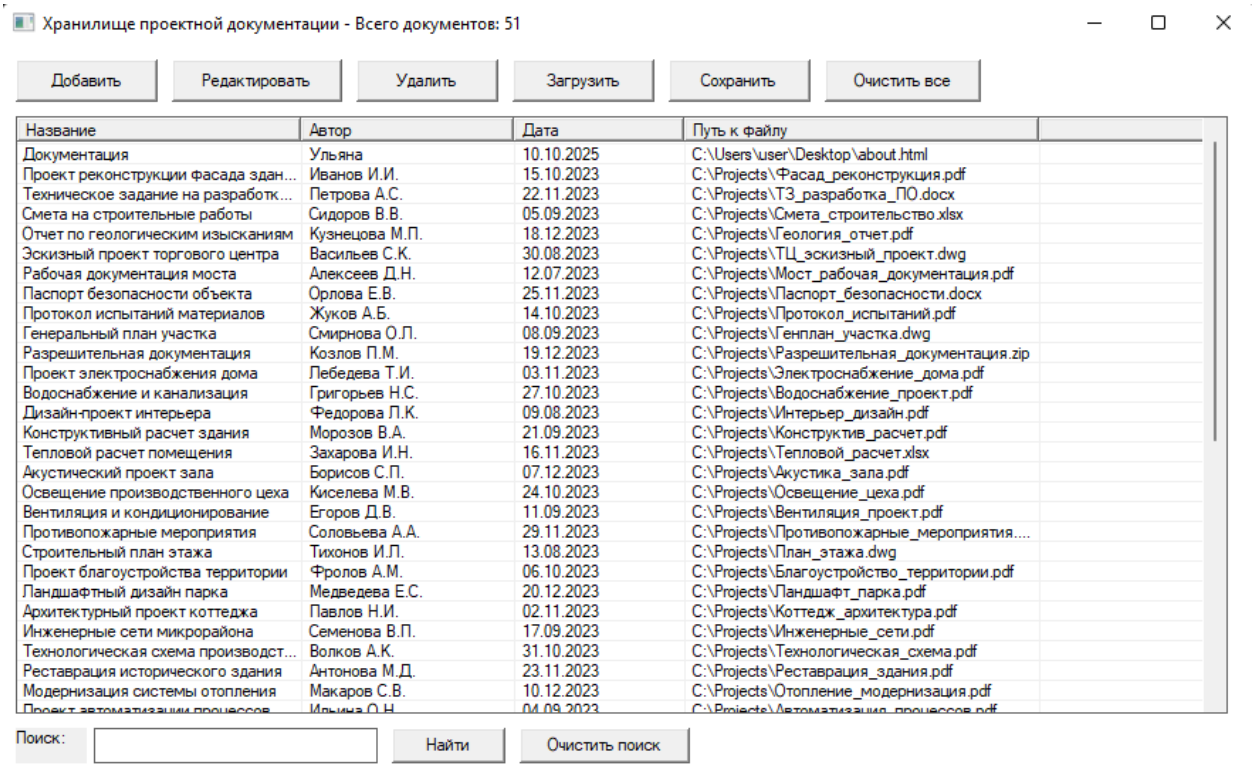

Хранилище проектной документации - Всего документов: 51
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документация
Ульяна
10.10.2025
C:\Users\user\Desktop\about.html
Проект реконструкции фасада здан...
Иванов И.И.
15.10.2023
C:\Projects\Фасад_реконструкция.pdf
Техническое задание на разработк...
Петрова А.С.
22.11.2023
C:\Projects\ТЗ_разработка_ПО.docx
Смета на строительные работы
Сидоров В.В.
05.09.2023
C:\Projects\Смета_строительство.xlsx
Отчет по геологическим изысканиям
Кузнецова М.П.
18.12.2023
C:\Projects\Геология_отчет.pdf
Эскизный проект торгового центра
Васильев С.К.
30.08.2023
C:\Projects\ТЦ_эскизный_проект.dwg
Рабочая документация моста
Алексеев Д.Н.
12.07.2023
C:\Projects\Мост_рабочая_документация.pdf
Паспорт безопасности объекта
Орлова Е.В.
25.11.2023
C:\Projects\Паспорт_безопасности.docx
Протокол испытаний материалов
Жуков А.Б.
14.10.2023
C:\Projects\Протокол_испытаний.pdf
Генеральный план участка
Смирнова О.Л.
08.09.2023
C:\Projects\Генплан_участка.dwg
Разрешительная документация
Козлов П.М.
19.12.2023
C:\Projects\Разрешительная_документация.zip
Проект электроснабжения дома
Лебедева Т.И.
03.11.2023
C:\Projects\Электроснабжение_дома.pdf
Водоснабжение и канализация
Григорьев Н.С.
27.10.2023
C:\Projects\Водоснабжение_проект.pdf
Дизайн-проект интерьера
Федорова Л.К.
09.08.2023
C:\Projects\Интерьер_дизайн.pdf
Конструктивный расчет здания
Морозов В.А.
21.09.2023
Тепловой расчет помещения
Захарова И.Н.
16.11.2023
Акустический проект зала
Борисов С.П.
07.12.2023
Освещение производственного цеха
Киселева М.В.
24.10.2023
Вентиляция и кондиционирование
Егоров Д.В.
11.09.2023
Противопожарные мероприятия
Соловьева А.А.
29.11.2023
Строительный план этажа
Тихонов И.Л.
13.08.2023
Проект благоустройства территории
Фролов А.М.
06.10.2023
Ландшафтный дизайн парка
Медведева Е.С.
20.12.2023
Архитектурный проект коттеджа
Павлов Н.И.
02.11.2023
Инженерные сети микрорайона
Семенова В.П.
17.09.2023
C:\Projects\Инженерные_сети.pdf
Технологическая схема производст...
Волков А.К.
31.10.2023
C:\Projects\Технологическая_схема.pdf
Реставрация исторического здания
Антонова М.Д.
23.11.2023
C:\Projects\Реставрация_здания.pdf
Модернизация системы отопления
Макаров С.В.
10.12.2023
C:\Projects\Отопление_модернизация.pdf
Загрузка завершена
Добавлено 50 новых документов
Всего документов: 51
OK
Поиск:
Найти
Очистить поиск

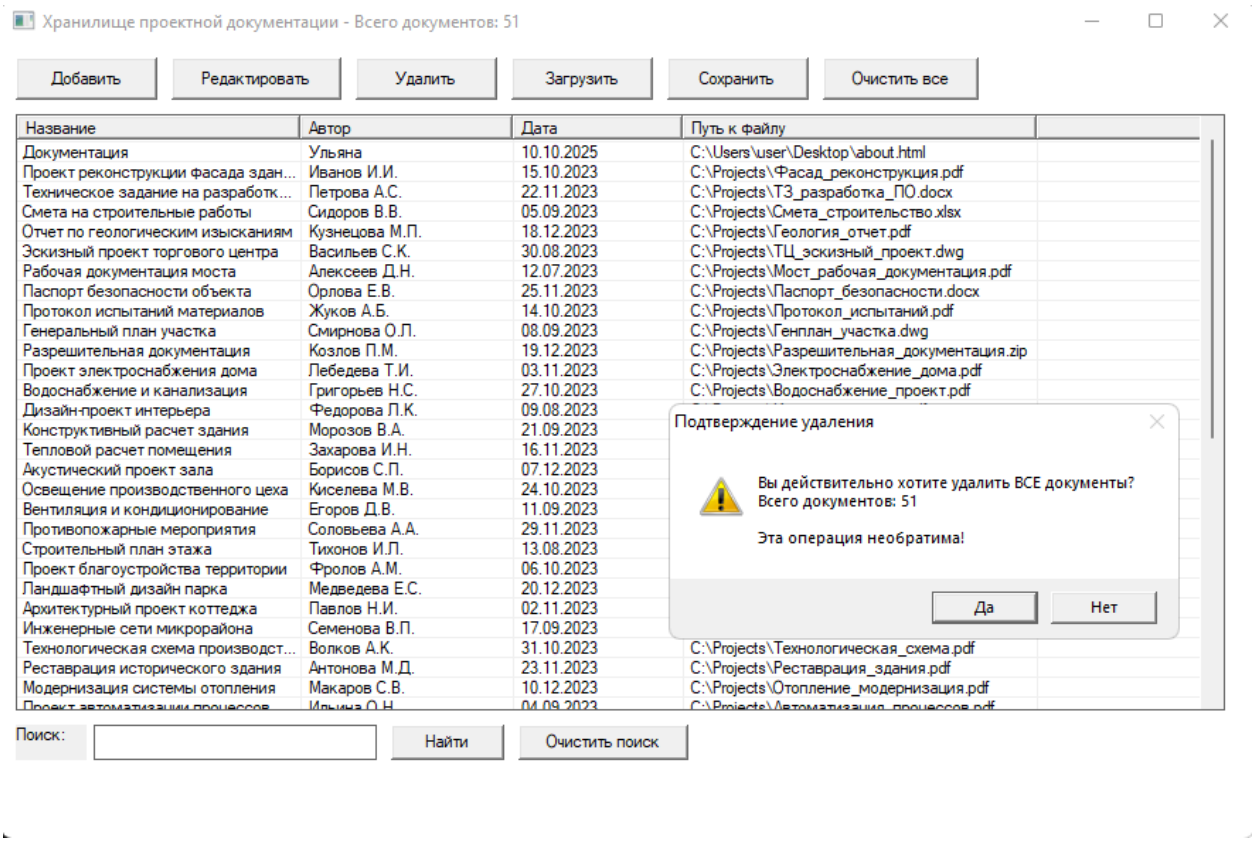

Хранилище проектной документации - Всего документов: 51
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документация
Ульяна
10.10.2025
C:\Users\user\Desktop\about.html
Проект реконструкции фасада здан...
Иванов И.И.
15.10.2023
C:\Projects\Фасад_реконструкция.pdf
Техническое задание на разработк...
Петрова А.С.
22.11.2023
C:\Projects\ТЗ_разработка_ПО.docx
Смета на строительные работы
Сидоров В.В.
05.09.2023
C:\Projects\Смета_строительство.xlsx
Отчет по геологическим изысканиям
Кузнецова М.П.
18.12.2023
C:\Projects\Геология_отчет.pdf
Эскизный проект торгового центра
Васильев С.К.
30.08.2023
C:\Projects\ТЦ_эскизный_проект.dwg
Рабочая документация моста
Алексеев Д.Н.
12.07.2023
C:\Projects\Мост_рабочая_документация.pdf
Паспорт безопасности объекта
Орлова Е.В.
25.11.2023
C:\Projects\Паспорт_безопасности.docx
Протокол испытаний материалов
Жуков А.Б.
14.10.2023
C:\Projects\Протокол_испытаний.pdf
Генеральный план участка
Смирнова О.Л.
08.09.2023
C:\Projects\Генплан_участка.dwg
Разрешительная документация
Козлов П.М.
19.12.2023
C:\Projects\Разрешительная_документация.zip
Проект электроснабжения дома
Лебедева Т.И.
03.11.2023
C:\Projects\Электроснабжение_дома.pdf
Водоснабжение и канализация
Григорьев Н.С.
27.10.2023
C:\Projects\Водоснабжение_проект.pdf
Дизайн-проект интерьера
Федорова Л.К.
09.08.2023
Конструктивный расчет здания
Морозов В.А.
21.09.2023
Тепловой расчет помещения
Захарова И.Н.
16.11.2023
Акустический проект зала
Борисов С.П.
07.12.2023
Освещение производственного цеха
Киселева М.В.
24.10.2023
Вентиляция и кондиционирование
Егоров Д.В.
11.09.2023
Противопожарные мероприятия
Соловьева А.А.
29.11.2023
Строительный план этажа
Тихонов И.Л.
13.08.2023
Проект благоустройства территории
Фролов А.М.
06.10.2023
Ландшафтный дизайн парка
Медведева Е.С.
20.12.2023
Архитектурный проект коттеджа
Павлов Н.И.
02.11.2023
Инженерные сети микрорайона
Семенова В.П.
17.09.2023
Технологическая схема производст...
Волков А.К.
31.10.2023
C:\Projects\Технологическая_схема.pdf
Реставрация исторического здания
Антонова М.Д.
23.11.2023
C:\Projects\Реставрация_здания.pdf
Модернизация системы отопления
Макаров С.В.
10.12.2023
C:\Projects\Отопление_модернизация.pdf
Подтверждение
Файл documents.txt уже существует.
7.08 КБ
Перезаписать файл?
Да
Нет
Поиск:
Найти
Очистить поиск

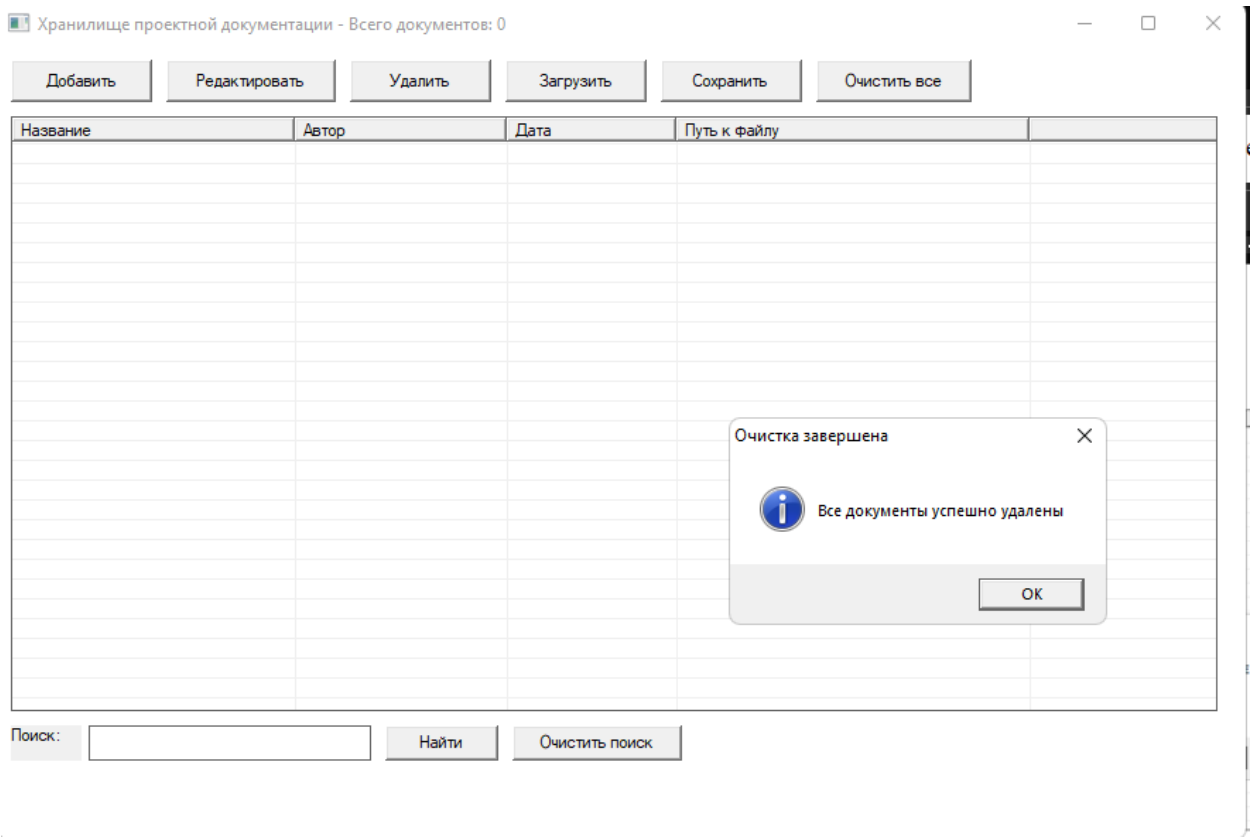

Хранилище проектной документации - Всего документов: 51
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документация
Ульяна
10.10.2025
C:\Users\user\Desktop\about.html
Проект реконструкции фасада здан...
Иванов И.И.
15.10.2023
C:\Projects\Фасад_реконструкция.pdf
Техническое задание на разработк...
Петрова А.С.
22.11.2023
C:\Projects\ТЗ_разработка_ПО.docx
Смета на строительные работы
Сидоров В.В.
05.09.2023
C:\Projects\Смета_строительство.xlsx
Отчет по геологическим изысканиям
Кузнецова М.П.
18.12.2023
C:\Projects\Геология_отчет.pdf
Эскизный проект торгового центра
Васильев С.К.
30.08.2023
C:\Projects\ТЦ_эскизный_проект.dwg
Рабочая документация моста
Алексеев Д.Н.
12.07.2023
C:\Projects\Мост_рабочая_документация.pdf
Паспорт безопасности объекта
Орлова Е.В.
25.11.2023
C:\Projects\Паспорт_безопасности.docx
Протокол испытаний материалов
Жуков А.Б.
14.10.2023
C:\Projects\Протокол_испытаний.pdf
Генеральный план участка
Смирнова О.Л.
08.09.2023
C:\Projects\Генплан_участка.dwg
Разрешительная документация
Козлов П.М.
19.12.2023
C:\Projects\Разрешительная_документация.zip
Проект электроснабжения дома
Лебедева Т.И.
03.11.2023
C:\Projects\Электроснабжение_дома.pdf
Водоснабжение и канализация
Григорьев Н.С.
27.10.2023
C:\Projects\Водоснабжение_проект.pdf
Дизайн-проект интерьера
Федорова Л.К.
09.08.2023
Конструктивный расчет здания
Морозов В.А.
21.09.2023
Тепловой расчет помещения
Захарова И.Н.
16.11.2023
Акустический проект зала
Борисов С.П.
07.12.2023
Освещение производственного цеха
Киселева М.В.
24.10.2023
Вентиляция и кондиционирование
Егоров Д.В.
11.09.2023
Противопожарные мероприятия
Соловьева А.А.
29.11.2023
Строительный план этажа
Тихонов И.Л.
13.08.2023
Проект благоустройства территории
Фролов А.М.
06.10.2023
Ландшафтный дизайн парка
Медведева Е.С.
20.12.2023
Архитектурный проект коттеджа
Павлов Н.И.
02.11.2023
Инженерные сети микрорайона
Семенова В.П.
17.09.2023
C:\Projects\Инженерные_сети.pdf
Технологическая схема производст...
Волков А.К.
31.10.2023
C:\Projects\Технологическая_схема.pdf
Реставрация исторического здания
Антонова М.Д.
23.11.2023
C:\Projects\Реставрация_здания.pdf
Модернизация системы отопления
Макаров С.В.
10.12.2023
C:\Projects\Отопление_модернизация.pdf
Сохранение завершена
Успешно сохранено 51 документов в documents.txt
OK
Поиск:
Найти
Очистить поиск


Хранилище проектной документации - Найдено: 4 (Всего: 51)
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документация
Ульяна
10.10.2025
C:\Users\user\Desktop\about.html
Рабочая документация моста
Алексеев Д.Н.
12.07.2023
C:\Projects\Мост_рабочая_документация.pdf
Разрешительная документация
Козлов П.М.
19.12.2023
C:\Projects\Разрешительная_документация.zip
Тендерная документация
Степанов А.В.
05.12.2023
C:\Projects\Тендерная_документация.zip
Поиск:
Документация
Найти
Очистить поиск


Хранилище проектной документации - Найдено: 3 (Всего: 51)
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документация
Ульяна
10.10.2025
C:\Users\user\Desktop\about.html
Рабочая документация моста
Алексеев Д.Н.
12.07.2023
C:\Projects\Мост_рабочая_документация.pdf
Тендерная документация
Степанов А.В.
05.12.2023
C:\Projects\Тендерная_документация.zip
Поиск:
Дакументация
Найти
Очистить поиск


Хранилище проектной документации - Всего документов: 51
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документация
Ульяна
10.10.2025
C:\Users\user\Desktop\about.html
Проект реконструкции фасада здан...
Иванов И.И.
15.10.2023
C:\Projects\Фасад_реконструкция.pdf
Техническое задание на разработк...
Петрова А.С.
22.11.2023
C:\Projects\ТЗ_разработка_ПО.docx
Смета на строительные работы
Сидоров В.В.
05.09.2023
C:\Projects\Смета_строительство.xlsx
Отчет по геологическим изысканиям
Кузнецова М.П.
18.12.2023
C:\Projects\Геология_отчет.pdf
Эскизный проект торгового центра
Васильев С.К.
30.08.2023
C:\Projects\ТЦ_эскизный_проект.dwg
Рабочая документация моста
Алексеев Д.Н.
12.07.2023
C:\Projects\Мост_рабочая_документация.pdf
Паспорт безопасности объекта
Орлова Е.В.
25.11.2023
C:\Projects\Паспорт_безопасности.docx
Протокол испытаний материалов
Жуков А.Б.
14.10.2023
C:\Projects\Протокол_испытаний.pdf
Генеральный план участка
Смирнова О.Л.
08.09.2023
C:\Projects\Генплан_участка.dwg
Разрешительная документация
Козлов П.М.
19.12.2023
C:\Projects\Разрешительная_документация.zip
Проект электроснабжения дома
Лебедева Т.И.
03.11.2023
C:\Projects\Электроснабжение_дома.pdf
Водоснабжение и канализация
Григорьев Н.С.
27.10.2023
C:\Projects\Водоснабжение_проект.pdf
Дизайн-проект интерьера
Федорова Л.К.
09.08.2023
C:\Projects\Интерьер_дизайн.pdf
Конструктивный расчет здания
Морозов В.А.
21.09.2023
C:\Projects\Конструктив_расчет.pdf
Тепловой расчет помещения
Захарова И.Н.
16.11.2023
C:\Projects\Тепловой_расчет.xlsx
Акустический проект зала
Борисов С.П.
07.12.2023
C:\Projects\Акустика_зала.pdf
Освещение производственного цеха
Киселева М.В.
24.10.2023
C:\Projects\Освещение_цеха.pdf
Вентиляция и кондиционирование
Егоров Д.В.
11.09.2023
C:\Projects\Вентиляция_проект.pdf
Противопожарные мероприятия
Соловьева А.А.
29.11.2023
C:\Projects\Противопожарные_мероприятия....
Строительный план этажа
Тихонов И.П.
13.08.2023
C:\Projects\План_этажа.dwg
Проект благоустройства территории
Фролов А.М.
06.10.2023
C:\Projects\Благоустройство_территории.pdf
Ландшафтный дизайн парка
Медведева Е.С.
20.12.2023
C:\Projects\Ландшафт_парка.pdf
Архитектурный проект коттеджа
Павлов Н.И.
02.11.2023
C:\Projects\Коттедж_архитектура.pdf
Инженерные сети микрорайона
Семенова В.П.
17.09.2023
C:\Projects\Инженерные_сети.pdf
Технологическая схема производст...
Волков А.К.
31.10.2023
C:\Projects\Технологическая_схема.pdf
Реставрация исторического здания
Антонова М.Д.
23.11.2023
C:\Projects\Реставрация_здания.pdf
Модернизация системы отопления
Макаров С.В.
10.12.2023
C:\Projects\Отопление_модернизация.pdf
Поиск:
Найти
Очистить поиск


Хранилище проектной документации - Всего документов: 51
Добавить
Редактировать
Удалить
Загрузить
Сохранить
Очистить все
Название
Автор
Дата
Путь к файлу
Документация
Ульяна
10.10.2025
C:\Users\user\Desktop\about.html
Проект реконструкции фасада здан...
Иванов И.И.
15.10.2023
C:\Projects\Фасад_реконструкция.pdf
Техническое задание на разработк...
Петрова А.С.
22.11.2023
C:\Projects\ТЗ_разработка_ПО.docx
Смета на строительные работы
Сидоров В.В.
05.09.2023
C:\Projects\Смета_строительство.xlsx
Отчет по геологическим изысканиям
Кузнецова М.П.
18.12.2023
C:\Projects\Геология_отчет.pdf
Эскизный проект торгового центра
Васильев С.К.
30.08.2023
C:\Projects\ТЦ_эскизный_проект.dwg
Рабочая документация моста
Алексеев Д.Н.
12.07.2023
C:\Projects\Мост_рабочая_документация.pdf
Паспорт безопасности объекта
Орлова Е.В.
25.11.2023
C:\Projects\Паспорт_безопасности.docx
Протокол испытаний материалов
Жуков А.Б.
14.10.2023
C:\Projects\Протокол_испытаний.pdf
Генеральный план участка
Смирнова О.Л.
08.09.2023
C:\Projects\Генплан_участка.dwg
Разрешительная документация
Козлов П.М.
19.12.2023
C:\Projects\Разрешительная_документация.zip
Проект электроснабжения дома
Лебедева Т.И.
03.11.2023
C:\Projects\Электроснабжение_дома.pdf
Водоснабжение и канализация
Григорьев Н.С.
27.10.2023
C:\Projects\Водоснабжение_проект.pdf
Дизайн-проект интерьера
Федорова Л.К.
09.08.2023
Конструктивный расчет здания
Морозов В.А.
21.09.2023
Тепловой расчет помещения
Захарова И.Н.
16.11.2023
Акустический проект зала
Борисов С.П.
07.12.2023
Освещение производственного цеха
Киселева М.В.
24.10.2023
Вентиляция и кондиционирование
Егоров Д.В.
11.09.2023
Противопожарные мероприятия
Соловьева А.А.
29.11.2023
Строительный план этажа
Тихонов И.П.
13.08.2023
Проект благоустройства территории
Фролов А.М.
06.10.2023
Ландшафтный дизайн парка
Медведева Е.С.
20.12.2023
Архитектурный проект коттеджа
Павлов Н.И.
02.11.2023
Инженерные сети микрорайона
Семенова В.П.
17.09.2023
Технологическая схема производст...
Волков А.К.
31.10.2023
C:\Projects\Технологическая_схема.pdf
Реставрация исторического здания
Антонова М.Д.
23.11.2023
C:\Projects\Реставрация_здания.pdf
Модернизация системы отопления
Макаров С.В.
10.12.2023
C:\Projects\Отопление_модернизация.pdf
Подтверждение удаления
Вы действительно хотите удалить ВСЕ документы?
Всего документов: 51
Эта операция необратима!
Да
Нет
Поиск:
Найти
Очистить поиск



5.5 Работа с программой

5.5.1 Добавление документа

Шаги:

1. Нажать кнопку **"Добавить"**
2. Заполнить поля в диалоговом окне:
   - **Название документа** (обязательно)
   - **Автор** (необязательно)
   - **Дата** в формате ДД.ММ.ГГГГ
   - **Путь к файлу** (обязательно)
3. Нажать кнопку **"Обзор..."** для выбора файла
4. Нажать **"ОК"** для сохранения

Проверки:

- Название не должно быть пустым
- Файл должен существовать (предупреждение если нет)
- Дата проверяется на корректность формата

5.5.2 Редактирование документа

Шаги:

1. Выбрать документ в таблице
2. Нажать кнопку **"Редактировать"** (или двойной щелчок)
3. Внести изменения в диалоговом окне
4. Нажать **"ОК"** для сохранения

Особенности:

- Сохраняются все поля документа
- Автоматическое обновление триграмм для поиска
- Проверка на дубликаты

5.5.3 Удаление документа

Шаги:

1. Выбрать документ в таблице
2. Нажать кнопку **"Удалить"**
3. Подтвердить удаление в диалоговом окне

Меры предосторожности:

- Запрос подтверждения перед удалением
- Удаление только из базы данных, файлы не удаляются
- Возможность отмены операции

5.5.4 Поиск документов

Шаги:

1. Ввести поисковый запрос в поле "Поиск"
2. Нажать кнопку **"Найти"** или клавишу Enter

Особенности поиска:

- **Нечеткий поиск**: находит документы с опечатками
- **По всем полям**: ищет в названии, авторе, дате, пути
- **Триграммный алгоритм**: устойчив к небольшим различиям

Примеры запросов:

- "проект дома" – найдет "Проект жилого дома"
- "иванов" – найдет документы автора "Иванов И.И."
- "2023" – найдет документы за 2023 год

5.5.5 Загрузка данных из файла

Шаги:

1. Нажать кнопку **"Загрузить"**
2. Выбрать режим загрузки:
   - **Добавить к существующим** – объединение данных
   - **Заменить все документы** – полная замена
3. Программа загрузит данные из documents.txt

Формат файла:

- UTF-8 кодировка
- Разделитель полей: точка с запятой
- Пример строки: Проект дома;Иванов И.И.;12.12.2023;C:\Projects\house.pdf

5.5.6 Сохранение данных в файл

Шаги:

1. Нажать кнопку **"Сохранить"**
2. Подтвердить перезапись файла (если существует)
3. Данные сохраняются в documents.txt

Особенности:

- Автоматическое устранение дубликатов
- Сохранение в формате UTF-8 с BOM
- Создание резервной копии при необходимости

5.5.7 Очистка всех документов

Шаги:

1. Нажать кнопку **"Очистить все"**
2. Подтвердить операцию в диалоговом окне
3. Все документы будут удалены из базы данных

Внимание: операция необратима, файлы документов не удаляются.

5.6 Описание контрольного примера

5.6.1 Подготовка тестовых данных

Создать файл test_documents.txt с содержимым:

Проект жилого дома;Иванов И.И.;15.01.2023;C:\Projects\house1.pdf
Техническое задание;Петров П.П.;20.02.2023;C:\Docs\specification.docx
Смета расходов;Сидоров С.С.;10.03.2023;C:\Finance\estimate.xlsx

5.6.2 Последовательность тестирования

Шаг 1: Загрузка данных

1. Запустить программу
2. Нажать "Загрузить"
3. Выбрать "Заменить все документы"
4. Проверить: в таблице должно отобразиться 3 документа

Шаг 2: Добавление документа

1. Нажать "Добавить"
2. Заполнить поля:
   - Название: "Архитектурный план"
   - Автор: "Васильев В.В."
   - Дата: "25.03.2023"
   - Путь: "C:\Plans\architecture.dwg"
3. Нажать "ОК"
4. Проверить: в таблице должно быть 4 документа

Шаг 3: Поиск документов

1. В поле поиска ввести: "проект"
2. Нажать "Найти"
3. Проверить: должен найтись документ "Проект жилого дома"
4. Ввести: "2023"
5. Проверить: должны найтись все 4 документа

Шаг 4: Редактирование документа

1. Выбрать документ "Техническое задание"
2. Нажать "Редактировать"
3. Изменить автора на "Петров Петр Петрович"
4. Нажать "ОК"
5. Проверить: изменения должны отобразиться в таблице

Шаг 5: Удаление документа

1. Выбрать документ "Смета расходов"
2. Нажать "Удалить"
3. Подтвердить удаление
4. Проверить: в таблице должно остаться 3 документа

Шаг 6: Сохранение данных

1. Нажать "Сохранить"
2. Подтвердить перезапись файла
3. Проверить файл documents.txt: должен содержать 3 документа

Шаг 7: Очистка данных

1. Нажать "Очистить все"
2. Подтвердить операцию
3. Проверить: таблица должна быть пустой

5.7 Возможные ошибки и способы их устранения

5.7.1 Ошибки при запуске программы

Таблица 1. Ошибки при запуске программы

Ошибка	Причина	Решение
"Не удалось запустить программу"	Исполняемый файл повреждён или скопирован не полностью	Пересобрать программу в Code::Blocks (Build → Rebuild, цель Release)
"Недостаточно памяти"	Нехватка оперативной памяти	Закрыть другие программы, увеличить файл подкачки
"Отказано в доступе"	Нет прав на запись в папку	Запустить от имени администратора или изменить папку

5.7.2 Ошибки при работе с документами

Таблица 2. Ошибки при работе с документами

Ошибка	Причина	Решение
"Название документа не может быть пустым"	Пустое поле названия	Ввести название документа
"Файл не существует"	Указан неверный путь	Проверить путь, использовать "Обзор..."
"Ошибка при сохранении файла"	Нет прав на запись	Проверить права доступа к файлу
"Неверный формат даты"	Дата в неправильном формате	Использовать формат ДД.ММ.ГГГГ

5.7.3 Ошибки при загрузке/сохранении

Таблица 3. Ошибки при загрузке/сохранении

Ошибка	Причина	Решение
"Файл не найден"	Файл documents.txt отсутствует	Создать файл через меню "Сохранить"
"Ошибка чтения файла"	Поврежденный файл	Восстановить из резервной копии

"Неверный формат файла"	Неправильная кодировка	Убедиться, что файл в UTF-8

5.7.4 Общие рекомендации

1. **Регулярное сохранение**: часто сохраняйте данные
2. **Резервное копирование**: создавайте копии файла documents.txt
3. **Проверка файлов**: убедитесь, что файлы документов существуют
4. **Валидация данных**: проверяйте корректность вводимых данных

6 ТЕСТИРОВАНИЕ И ОТЛАДКА ПРОГРАММЫ

6.1 Методология тестирования

6.1.1 Виды тестирования

1. **Модульное тестирование**:
   - Тестирование отдельных классов и функций
   - Проверка методов класса Document
   - Тестирование алгоритмов поиска
2. **Интеграционное тестирование**:
   - Тестирование взаимодействия модулей
   - Проверка работы GUI с бизнес-логикой
   - Тестирование файловых операций
3. **Системное тестирование**:
   - Тестирование всей системы в целом
   - Проверка соответствия требованиям
   - Тестирование производительности
4. **Приемочное тестирование**:
   - Тестирование пользователем
   - Проверка удобства использования
   - Оценка соответствия ожиданиям

6.1.2 Инструменты тестирования

1. **Отладчик: GDB (встроенный отладчик Code::Blocks)**
2. **Профилировщик: gprof (плагин Code Profiler в Code::Blocks)**
3. **Статический анализ**: PVS-Studio
4. **Ручное тестирование**: тестовые сценарии

6.2 Тестовые сценарии

6.2.1 Сценарий 1: Работа с документами

Цель: проверка CRUD операций.

Шаги:

1. Добавить 10 различных документов
2. Отредактировать 5 документов
3. Удалить 3 документа
4. Проверить целостность оставшихся документов

Ожидаемый результат: все операции выполняются корректно.

6.2.2 Сценарий 2: Поиск документов

Цель: проверка алгоритма поиска.

Шаги:

1. Добавить документы с разными названиями
2. Выполнить поиск по точным совпадениям
3. Выполнить поиск с опечатками
4. Проверить релевантность результатов

Ожидаемый результат: поиск находит все релевантные документы.

6.2.3 Сценарий 3: Работа с файлами

Цель: проверка операций ввода/вывода.

Шаги:

1. Сохранить данные в файл
2. Загрузить данные из файла
3. Проверить целостность загруженных данных
4. Проверить работу с большими файлами (10 000 записей)

Ожидаемый результат: данные сохраняются и загружаются без потерь.

6.3 Результаты тестирования

6.3.1 Функциональное тестирование

Таблица 3. Функциональное тестирование

Функция	**Результат**	**Примечания**
Добавление документа	Успешно	Проверены все валидации
Редактирование документа	Успешно	Корректное обновление данных
Удаление документа	Успешно	Подтверждение перед удалением
Поиск документов	Успешно	Работает нечеткий поиск
Сохранение в файл	Успешно	UTF-8 кодировка, устранение дубликатов

Загрузка из файла	Успешно	Обработка ошибок формата
Очистка всех документов	Успешно	Подтверждение операции

6.3.2 Нефункциональное тестирование

Характеристика	Результат	Метрика
Производительность	Приемлема	1000 документов загружается за < 1 сек
Надежность	Высокая	Обработка всех исключительных ситуаций
Удобство использования	Хорошее	Интуитивный интерфейс
Совместимость	Полная	Windows 7-11, x86/x64

6.3.3 Найденные и исправленные ошибки

1. **Ошибка**: утечка памяти при удалении документов
   Решение: добавлен корректный вызов деструкторов
2. **Ошибка**: некорректная обработка UTF-8 BOM
   Решение: добавлена явная проверка и пропуск BOM
3. **Ошибка**: падение при пустом файле данных
   Решение: добавлена проверка размера файла
4. **Ошибка**: неверная работа поиска с короткими словами
   Решение: добавлена обработка коротких запросов

6.4 Отладка программы

6.4.1 Используемые методы отладки

1. **Логирование**:

```
#ifdef _DEBUG
OutputDebugStringW(L"Функция вызвана\n");
#endif
```

2. **Точки останова**: установка в критических местах кода
3. **Пошаговое выполнение**: проверка потока выполнения
4. **Просмотр переменных**: мониторинг состояния объектов

6.4.2 Типичные проблемы и решения

1. **Проблема**: некорректное отображение Unicode
   Решение: использование W-версий функций Windows API
2. **Проблема**: утечки ресурсов
   Решение: использование RAII, умных указателей
3. **Проблема**: блокировка GUI при долгих операциях
   Решение: асинхронное выполнение операций
4. **Проблема**: конфликты доступа к файлам
   Решение: блокировка файлов при записи

7 ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был разработан программный продукт «Хранилище проектной документации», предназначенный для автоматизации учета и управления строительной документацией.

7.1 Достигнутые результаты

1. **Полнофункциональная система**:
   - Реализованы все запланированные функции
   - Создан удобный графический интерфейс
   - Реализован эффективный алгоритм поиска
2. **Качество кода**:
   - Соблюдены принципы ООП
   - Использованы современные возможности C++17
   - Обеспечена надежность и безопасность
3. **Документация**:
   - Разработана полная техническая документация
   - Создано руководство пользователя
   - Подготовлены тестовые сценарии

7.2 Соответствие требованиям

Программа соответствует всем требованиям технического задания:

- Функциональные требования выполнены полностью
- Требования к интерфейсу соблюдены
- Требования к надежности реализованы
- Эргономические требования учтены

7.3 Научно-практическая значимость

1. **Научная значимость**:
   - Исследован и реализован алгоритм триграммного поиска
   - Разработана архитектура системы управления документами
   - Изучены особенности разработки под Windows API
2. **Практическая значимость**:
   - Программа может использоваться в строительных организациях
   - Сокращает время поиска документов
   - Упрощает управление документацией

7.4 Перспективы развития

1. **Доработки и улучшения**:

- Добавление сетевого режима работы
- Интеграция с облачными хранилищами
- Поддержка сканирования документов
- Добавление системы прав доступа

2. **Оптимизации**:

- Переход на базу данных SQLite
- Реализация полнотекстового поиска
- Добавление кэширования результатов

3. **Расширение функционала**:

- Поддержка версионности документов
- Добавление workflow документов
- Интеграция с электронной подписью

7.5 Выводы

Разработанный программный продукт представляет собой законченное решение для управления проектной документацией в строительных организациях. Программа успешно прошла все этапы тестирования и готова к использованию.

Основные преимущества системы:

- Простота использования
- Надежность работы
- Эффективный поиск документов
- Минимальные системные требования
- Бесплатное распространение

Работа продемонстрировала умение применять теоретические знания на практике, навыки объектно-ориентированного проектирования и программирования на C++.

8 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

8.1 Нормативные документы

1. ГОСТ 34.003-90 "Автоматизированные системы. Термины и определения"
2. ГОСТ 34.601-90 "Автоматизированные системы. Стадии создания"
3. ГОСТ 19.101-77 "Единая система программной документации. Виды программ и программных документов"
4. ГОСТ Р ИСО/МЭК 12207-2010 "Информационная технология. Процессы жизненного цикла программных средств"

8.2 Книги и учебные пособия

1. Страуструп Б. Язык программирования C++. – 4-е изд. – М.: ДМК Пресс, 2019. – 1136 с.
2. Петцольд Ч. Программирование для Windows 95. – М.: БИНОМ, 1996. – 848 с.
3. Рихтер Д. Windows для профессионалов. – СПб.: Питер, 2018. – 752 с.
4. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. – СПб.: Питер, 2020. – 366 с.
5. Макконнелл С. Совершенный код. – 2-е изд. – М.: Русская редакция, 2019. – 896 с.
6. Липпман С., Лажойе Ж., Му Б. Язык программирования C++. Базовый курс. – 5-е изд. – М.: Вильямс, 2021. – 1120 с.

8.3 Статьи и научные публикации

1. Zobel J., Moffat A. Inverted files for text search engines // ACM Computing Surveys. – 2006. – Vol. 38, No. 2.
2. Manning C.D., Raghavan P., Schütze H. Introduction to Information Retrieval. – Cambridge University Press, 2008.
3. Salton G., McGill M.J. Introduction to Modern Information Retrieval. – McGraw-Hill, 1986.
4. Ukkonen E. Approximate string matching with q-grams and maximal matches // Theoretical Computer Science. – 1992. – Vol. 92, No. 1. – P. 191-211.

8.4 Интернет-ресурсы

1. Microsoft Docs: Windows API Reference – https://docs.microsoft.com/en-us/windows/win32/api/
2. C++ Reference – https://en.cppreference.com/w/
3. Unicode Consortium – https://unicode.org/
4. Stack Overflow – https://stackoverflow.com/
5. GitHub – https://github.com/

8.5 Программное обеспечение

1. Code::Blocks 25.03 (компилятор MinGW-w64 GCC 14.2)
2. Заголовочные файлы и библиотеки Windows API из состава MinGW-w64
3. Git для контроля версий
4. Doxygen для генерации документации

ПРИЛОЖЕНИЯ

Приложение А. Листинг программы

```
#include <windows.h>
#include <commctrl.h>
#include <string>
#include <vector>
#include <fstream>
#include <sstream>
#include <codecvt>
#include <locale>
#include <algorithm>
#include <cctype>
#include <set>
#include <iomanip>
#include <map>
#include <unordered_set>

#ifndef IDC_STATIC
#define IDC_STATIC -1
#endif

#define IDC_LISTVIEW 1001
#define IDC_ADDBUTTON 1002
#define IDC_EDITBUTTON 1003
#define IDC_DELETEBUTTON 1004
#define IDC_LOADBUTTON 1005
#define IDC_SAVEBUTTON 1006
#define IDC_SEARCHEDIT 1007
#define IDC_SEARCHBUTTON 1008
#define IDC_CLEARBUTTON 1009          // Кнопка "Очистить все"
#define IDC_CLEARSEARCHBUTTON 1010    // Кнопка "Очистить поиск"

#define IDC_NAME_EDIT 101
#define IDC_AUTHOR_EDIT 102
#define IDC_DATE_EDIT 103
#define IDC_PATH_EDIT 104
#define IDC_BROWSE_BUTTON 105
#define IDC_OK_BUTTON 106
#define IDC_CANCEL_BUTTON 107

// Вспомогательные функции для конвертации
std::string WStringToUTF8(const std::wstring& wstr) {
    if (wstr.empty()) return "";
```

```
    int size_needed = WideCharToMultiByte(CP_UTF8, 0, &wstr[0],
(int)wstr.size(),
                                          NULL, 0, NULL, NULL);
    std::string strTo(size_needed, 0);
    WideCharToMultiByte(CP_UTF8, 0, &wstr[0], (int)wstr.size(),
                        &strTo[0], size_needed, NULL, NULL);
    return strTo;
}

std::wstring UTF8ToWString(const std::string& str) {
    if (str.empty()) return L"";
    int size_needed = MultiByteToWideChar(CP_UTF8, 0, &str[0], (int)str.size(),
                                          NULL, 0);
    std::wstring wstrTo(size_needed, 0);
    MultiByteToWideChar(CP_UTF8, 0, &str[0], (int)str.size(),
                        &wstrTo[0], size_needed);
    return wstrTo;
}

std::string WStringToString(const std::wstring& wstr) {
    if (wstr.empty()) return "";
    int size_needed = WideCharToMultiByte(CP_ACP, 0, &wstr[0],
(int)wstr.size(),
                                          NULL, 0, NULL, NULL);
    std::string strTo(size_needed, 0);
    WideCharToMultiByte(CP_ACP, 0, &wstr[0], (int)wstr.size(),
                        &strTo[0], size_needed, NULL, NULL);
    return strTo;
}

std::wstring StringToWString(const std::string& str) {
    if (str.empty()) return L"";
    int size_needed = MultiByteToWideChar(CP_ACP, 0, &str[0], (int)str.size(),
                                          NULL, 0);
    std::wstring wstrTo(size_needed, 0);
    MultiByteToWideChar(CP_ACP, 0, &str[0], (int)str.size(),
                        &wstrTo[0], size_needed);
    return wstrTo;
}

std::wstring TrimWString(const std::wstring& str) {
    size_t first = str.find_first_not_of(L" \t\n\r");
    if (first == std::wstring::npos) return L"";
    size_t last = str.find_last_not_of(L" \t\n\r");
```

```
    return str.substr(first, last - first + 1);
}

std::string TrimString(const std::string& str) {
    size_t first = str.find_first_not_of(" \t\n\r");
    if (first == std::string::npos) return "";
    size_t last = str.find_last_not_of(" \t\n\r");
    return str.substr(first, last - first + 1);
}

bool FileExists(const std::wstring& filename) {
    DWORD dwAttrib = GetFileAttributesW(filename.c_str());
    return (dwAttrib != INVALID_FILE_ATTRIBUTES &&
            !(dwAttrib & FILE_ATTRIBUTE_DIRECTORY));
}

// Функция для генерации триграмм из строки
std::unordered_set<std::wstring> GenerateTrigrams(const std::wstring& str) {
    std::unordered_set<std::wstring> trigrams;
    if (str.length() < 3) {
        // Для коротких строк добавляем саму строку как триграмму
        if (!str.empty()) {
            trigrams.insert(str);
        }
        return trigrams;
    }

    for (size_t i = 0; i <= str.length() - 3; ++i) {
        trigrams.insert(str.substr(i, 3));
    }
    return trigrams;
}

// Функция для вычисления коэффициента сходства по триграммам
(коэффициент Жаккара)
double CalculateSimilarity(const std::unordered_set<std::wstring>& trigrams1,
                           const std::unordered_set<std::wstring>& trigrams2) {
    if (trigrams1.empty() && trigrams2.empty()) return 1.0;
    if (trigrams1.empty() || trigrams2.empty()) return 0.0;

    int intersection = 0;
    for (const auto& trigram : trigrams1) {
        if (trigrams2.find(trigram) != trigrams2.end()) {
            intersection++;
```

```
        }
    }

    int union_size = trigrams1.size() + trigrams2.size() - intersection;
    return union_size > 0 ? static_cast<double>(intersection) / union_size : 0.0;
}

// Класс документа
class Document {
private:
    std::wstring name;
    std::wstring author;
    std::wstring date;
    std::wstring filePath;
    std::unordered_set<std::wstring> nameTrigrams;
    std::unordered_set<std::wstring> authorTrigrams;

public:
    Document(const std::wstring& n = L"", const std::wstring& a = L"",
             const std::wstring& d = L"", const std::wstring& f = L"")
        : name(TrimWString(n)), author(TrimWString(a)),
          date(TrimWString(d)), filePath(TrimWString(f)) {
        updateTrigrams();
    }

    void updateTrigrams() {
        nameTrigrams = GenerateTrigrams(name);
        authorTrigrams = GenerateTrigrams(author);
    }

    bool isValid() const {
        return !name.empty() && !filePath.empty();
    }

    std::wstring getName() const { return name; }
    std::wstring getAuthor() const { return author; }
    std::wstring getDate() const { return date; }
    std::wstring getFilePath() const { return filePath; }

    const std::unordered_set<std::wstring>& getNameTrigrams() const { return
nameTrigrams; }
    const std::unordered_set<std::wstring>& getAuthorTrigrams() const { return
authorTrigrams; }
```

```
    void setName(const std::wstring& n) {
        name = TrimWString(n);
        updateTrigrams();
    }
    void setAuthor(const std::wstring& a) {
        author = TrimWString(a);
        updateTrigrams();
    }
    void setDate(const std::wstring& d) { date = TrimWString(d); }
    void setFilePath(const std::wstring& f) { filePath = TrimWString(f); }

    // Для сравнения документов (для предотвращения дубликатов)
    bool operator==(const Document& other) const {
        return name == other.name &&
               author == other.author &&
               date == other.date &&
               filePath == other.filePath;
    }

    // Для хранения в set
    bool operator<(const Document& other) const {
        if (name != other.name) return name < other.name;
        if (author != other.author) return author < other.author;
        if (date != other.date) return date < other.date;
        return filePath < other.filePath;
    }

    std::string toString() const {
        std::string name_utf8 = WStringToUTF8(name);
        std::string author_utf8 = WStringToUTF8(author);
        std::string date_utf8 = WStringToUTF8(date);
        std::string filePath_utf8 = WStringToUTF8(filePath);

        return name_utf8 + ";" + author_utf8 + ";" + date_utf8 + ";" + filePath_utf8;
    }

    static Document fromString(const std::string& str) {
        std::string trimmed = TrimString(str);
        if (trimmed.empty()) return Document();

        size_t pos1 = trimmed.find(';');
        if (pos1 == std::string::npos) return Document(UTF8ToWString(trimmed));

        size_t pos2 = trimmed.find(';', pos1 + 1);
```

```
        if (pos2 == std::string::npos) return Document();

        size_t pos3 = trimmed.find(';', pos2 + 1);
        if (pos3 == std::string::npos) return Document();

        std::string name_utf8 = trimmed.substr(0, pos1);
        std::string author_utf8 = trimmed.substr(pos1 + 1, pos2 - pos1 - 1);
        std::string date_utf8 = trimmed.substr(pos2 + 1, pos3 - pos2 - 1);
        std::string filePath_utf8 = trimmed.substr(pos3 + 1);

        return Document(UTF8ToWString(name_utf8),
UTF8ToWString(author_utf8),
                        UTF8ToWString(date_utf8), UTF8ToWString(filePath_utf8));
    }
};

// Класс хранилища документов
class DocumentStorage {
private:
    std::vector<Document> documents;
    bool isSearchMode;
    std::vector<Document> searchResults;
    std::wstring lastSearchQuery;

public:
    DocumentStorage() : isSearchMode(false), lastSearchQuery(L"") {}

    void addDocument(const Document& doc) {
        if (doc.isValid()) {
            // Проверяем дубликаты
            if (std::find(documents.begin(), documents.end(), doc) ==
documents.end()) {
                documents.push_back(doc);
            }
        }
    }

    bool editDocument(size_t index, const Document& doc) {
        if (index < documents.size() && doc.isValid()) {
            documents[index] = doc;
            return true;
        }
        return false;
    }
```

```
    bool deleteDocument(size_t index) {
        if (isSearchMode) {
            // Удаляем из результатов поиска и из основного хранилища
            if (index < searchResults.size()) {
                Document docToDelete = searchResults[index];
                // Находим и удаляем из основного списка
                auto it = std::find(documents.begin(), documents.end(), docToDelete);
                if (it != documents.end()) {
                    documents.erase(it);
                }
                // Обновляем результаты поиска
                searchDocuments(lastSearchQuery);
                return true;
            }
        } else {
            if (index < documents.size()) {
                documents.erase(documents.begin() + index);
                return true;
            }
        }
        return false;
    }

    void searchDocuments(const std::wstring& keyword) {
        lastSearchQuery = keyword;
        searchResults.clear();

        if (keyword.empty()) {
            isSearchMode = false;
            return;
        }

        isSearchMode = true;

        // Генерируем триграммы для поискового запроса
        std::wstring lowerKeyword = keyword;
        std::transform(lowerKeyword.begin(), lowerKeyword.end(),
lowerKeyword.begin(), ::towlower);
        std::unordered_set<std::wstring> keywordTrigrams =
GenerateTrigrams(lowerKeyword);

        const double SIMILARITY_THRESHOLD = 0.3; // 30% сходства
```

```
    for (const auto& doc : documents) {
        // Проверяем по названию
        std::wstring docName = doc.getName();
        std::transform(docName.begin(), docName.end(), docName.begin(),
::towlower);

        std::unordered_set<std::wstring> docNameTrigrams =
GenerateTrigrams(docName);
        double nameSimilarity = CalculateSimilarity(keywordTrigrams,
docNameTrigrams);

        // Проверяем по автору
        std::wstring docAuthor = doc.getAuthor();
        std::transform(docAuthor.begin(), docAuthor.end(), docAuthor.begin(),
::towlower);

        std::unordered_set<std::wstring> docAuthorTrigrams =
GenerateTrigrams(docAuthor);
        double authorSimilarity = CalculateSimilarity(keywordTrigrams,
docAuthorTrigrams);

        // Также проверяем обычное вхождение для коротких запросов
        bool exactMatch = (keyword.length() <= 3 &&
                           (docName.find(lowerKeyword) != std::wstring::npos ||
                            docAuthor.find(lowerKeyword) != std::wstring::npos));

        // Также проверяем по дате и пути
        std::wstring docDate = doc.getDate();
        std::transform(docDate.begin(), docDate.end(), docDate.begin(),
::towlower);

        std::wstring docPath = doc.getFilePath();
        std::transform(docPath.begin(), docPath.end(), docPath.begin(),
::towlower);

        // Добавляем в результаты, если превышен порог или есть точное
совпадение
        if (nameSimilarity >= SIMILARITY_THRESHOLD ||
            authorSimilarity >= SIMILARITY_THRESHOLD ||
            exactMatch ||
            docName.find(lowerKeyword) != std::wstring::npos ||
            docAuthor.find(lowerKeyword) != std::wstring::npos ||
            docDate.find(lowerKeyword) != std::wstring::npos ||
            docPath.find(lowerKeyword) != std::wstring::npos) {
```

```
            searchResults.push_back(doc);
        }
    }
}

void clearSearch() {
    isSearchMode = false;
    searchResults.clear();
    lastSearchQuery = L"";
}

void clearAllDocuments() {
    documents.clear();
    clearSearch();
}

const std::vector<Document>& getCurrentDocuments() const {
    return isSearchMode ? searchResults : documents;
}

Document getDocument(size_t index) const {
    const std::vector<Document>& currentDocs = isSearchMode ? searchResults
: documents;
    if (index < currentDocs.size()) {
        return currentDocs[index];
    }
    return Document();
}

Document getDocumentFromStorage(size_t index) const {
    if (index < documents.size()) {
        return documents[index];
    }
    return Document();
}

size_t getDocumentCount() const {
    return isSearchMode ? searchResults.size() : documents.size();
}

size_t getTotalDocumentCount() const {
    return documents.size();
}
```

```
bool isInSearchMode() const {
    return isSearchMode;
}

std::wstring getLastSearchQuery() const {
    return lastSearchQuery;
}

bool saveToFile(const std::wstring& filename) const {
    if (documents.empty()) {
        return false;
    }

    std::string filename_str = WStringToString(filename);
    std::ofstream file(filename_str, std::ios::binary);
    if (!file.is_open()) {
        return false;
    }

    // Пишем UTF-8 BOM
    unsigned char bom[] = {0xEF, 0xBB, 0xBF};
    file.write(reinterpret_cast<const char*>(bom), sizeof(bom));

    // Используем set для устранения дубликатов при сохранении
    std::set<Document> uniqueDocs(documents.begin(), documents.end());

    for (const auto& doc : uniqueDocs) {
        std::string line = doc.toString();
        file << line << "\r\n";
    }

    file.close();
    return true;
}

bool loadFromFile(const std::wstring& filename, bool clearExisting = true) {
    std::string filename_str = WStringToString(filename);
    std::ifstream file(filename_str, std::ios::binary);
    if (!file.is_open()) {
        return false;
    }

    std::vector<Document> loadedDocuments;
    if (!clearExisting) {
```

```
    loadedDocuments = documents;
}

file.seekg(0, std::ios::end);
std::streamsize fileSize = file.tellg();
file.seekg(0, std::ios::beg);

if (fileSize == 0) {
    file.close();
    return true;
}

std::string line;
int loadedCount = 0;
int errorCount = 0;

if (fileSize >= 3) {
    unsigned char bom[3];
    file.read(reinterpret_cast<char*>(bom), 3);
    if (!(bom[0] == 0xEF && bom[1] == 0xBB && bom[2] == 0xBF)) {
        file.seekg(0, std::ios::beg);
    }
}

while (std::getline(file, line)) {
    if (!line.empty() && line.back() == '\r') {
        line.pop_back();
    }

    if (line.size() >= 3 &&
        static_cast<unsigned char>(line[0]) == 0xEF &&
        static_cast<unsigned char>(line[1]) == 0xBB &&
        static_cast<unsigned char>(line[2]) == 0xBF) {
        line = line.substr(3);
    }

    std::string trimmed = TrimString(line);
    if (trimmed.empty()) {
        continue;
    }

    try {
        Document doc = Document::fromString(trimmed);
        if (doc.isValid()) {
```

```
                // Проверяем дубликаты
                if (std::find(loadedDocuments.begin(), loadedDocuments.end(), doc)
== loadedDocuments.end()) {
                    loadedDocuments.push_back(doc);
                    loadedCount++;
                }
            } else {
                errorCount++;
            }
        } catch (...) {
            errorCount++;
            continue;
        }
    }

    file.close();

    if (loadedCount > 0) {
        documents = loadedDocuments;
        clearSearch(); // Сбрасываем поиск после загрузки

        if (errorCount > 0) {
            std::wstring message = L"Загружено " + std::to_wstring(loadedCount) +
                                   L" документов.\n" +
                                   L"Пропущено " + std::to_wstring(errorCount) +
                                   L" строк с ошибками.";
            MessageBoxW(NULL, message.c_str(), L"Загрузка с
предупреждениями",
                        MB_OK | MB_ICONWARNING);
        }
        return true;
    }

    return false;
  }
};

// Глобальные объекты
DocumentStorage storage;
HWND hListView;
HWND hSearchEdit;
HWND hMainWindow = NULL;
HWND hDialog = NULL;
bool isEditMode = false;
```

```
int editIndex = -1;

LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
LRESULT CALLBACK DialogProc(HWND, UINT, WPARAM, LPARAM);
void InitListView(HWND hwndListView);
void RefreshListView(HWND hwndListView);
void ShowDocumentDialog(HWND hParent, bool edit, int index);
bool ValidateDocumentFields(const std::wstring& name, const std::wstring&
path);
std::wstring GetFileSizeString(const std::wstring& filename);

// Инициализация ListView
void InitListView(HWND hwndListView) {
   ListView_SetExtendedListViewStyle(hwndListView,
      LVS_EX_FULLROWSELECT | LVS_EX_GRIDLINES |
LVS_EX_DOUBLEBUFFER);

   LVCOLUMNW lvc;
   ZeroMemory(&lvc, sizeof(LVCOLUMNW));
   lvc.mask = LVCF_TEXT | LVCF_WIDTH | LVCF_FMT;
   lvc.fmt = LVCFMT_LEFT;

   const wchar_t* headers[] = {L"Название", L"Автор", L"Дата", L"Путь к
файлу"};
   int widths[] = {200, 150, 120, 250};

   for (int i = 0; i < 4; i++) {
      lvc.pszText = const_cast<wchar_t*>(headers[i]);
      lvc.cx = widths[i];
      SendMessageW(hwndListView, LVM_INSERTCOLUMNW, i,
(LPARAM)&lvc);
   }
}

// Обновление ListView
void RefreshListView(HWND hwndListView) {
   SendMessageW(hwndListView, LVM_DELETEALLITEMS, 0, 0);

   const std::vector<Document>& docs = storage.getCurrentDocuments();

   for (size_t i = 0; i < docs.size(); ++i) {
      LVITEMW lvi;
      ZeroMemory(&lvi, sizeof(LVITEMW));
      lvi.mask = LVIF_TEXT;
```

```
        lvi.iItem = (int)i;

        lvi.iSubItem = 0;
        std::wstring name = docs[i].getName();
        lvi.pszText = const_cast<LPWSTR>(name.c_str());
        SendMessageW(hwndListView, LVM_INSERTITEMW, 0,
(LPARAM)&lvi);

        lvi.iSubItem = 1;
        std::wstring author = docs[i].getAuthor();
        lvi.pszText = const_cast<LPWSTR>(author.c_str());
        SendMessageW(hwndListView, LVM_SETITEMW, 0, (LPARAM)&lvi);

        lvi.iSubItem = 2;
        std::wstring date = docs[i].getDate();
        lvi.pszText = const_cast<LPWSTR>(date.c_str());
        SendMessageW(hwndListView, LVM_SETITEMW, 0, (LPARAM)&lvi);

        lvi.iSubItem = 3;
        std::wstring filePath = docs[i].getFilePath();
        lvi.pszText = const_cast<LPWSTR>(filePath.c_str());
        SendMessageW(hwndListView, LVM_SETITEMW, 0, (LPARAM)&lvi);
    }

    // Обновляем заголовок окна
    std::wstring title = L"Хранилище проектной документации";
    if (storage.isInSearchMode()) {
        title += L" - Найдено: " + std::to_wstring(storage.getDocumentCount()) +
                 L" (Всего: " + std::to_wstring(storage.getTotalDocumentCount()) +
L")";
    } else {
        title += L" - Всего документов: " +
std::to_wstring(storage.getTotalDocumentCount());
    }
    SetWindowTextW(hMainWindow, title.c_str());
}

bool ValidateDocumentFields(const std::wstring& name, const std::wstring& path)
{
    if (name.empty()) {
        MessageBoxW(NULL, L"Название документа не может быть пустым",
                    L"Ошибка валидации", MB_OK | MB_ICONERROR);
        return false;
    }
```

```
    if (path.empty()) {
        MessageBoxW(NULL, L"Путь к файлу не может быть пустым",
                L"Ошибка валидации", MB_OK | MB_ICONERROR);
        return false;
    }

    if (!FileExists(path)) {
        if (MessageBoxW(NULL, L"Указанный файл не
существует.\nПродолжить сохранение?",
                    L"Предупреждение", MB_YESNO | MB_ICONWARNING) !=
IDYES) {
            return false;
        }
    }

    return true;
}

std::wstring GetFileSizeString(const std::wstring& filename) {
    if (!FileExists(filename)) {
        return L"Файл не найден";
    }

    HANDLE hFile = CreateFileW(filename.c_str(), GENERIC_READ,
                        FILE_SHARE_READ, NULL, OPEN_EXISTING,
                        FILE_ATTRIBUTE_NORMAL, NULL);

    if (hFile == INVALID_HANDLE_VALUE) {
        return L"Ошибка доступа";
    }

    LARGE_INTEGER fileSize;
    if (!GetFileSizeEx(hFile, &fileSize)) {
        CloseHandle(hFile);
        return L"Ошибка определения размера";
    }

    CloseHandle(hFile);

    if (fileSize.QuadPart < 1024) {
        return std::to_wstring(fileSize.QuadPart) + L" байт";
    } else if (fileSize.QuadPart < 1024 * 1024) {
        double kb = fileSize.QuadPart / 1024.0;
```

```
        std::wstringstream ws;
        ws << std::fixed << std::setprecision(2) << kb << L" КБ";
        return ws.str();
    } else if (fileSize.QuadPart < 1024 * 1024 * 1024) {
        double mb = fileSize.QuadPart / (1024.0 * 1024.0);
        std::wstringstream ws;
        ws << std::fixed << std::setprecision(2) << mb << L" МБ";
        return ws.str();
    } else {
        double gb = fileSize.QuadPart / (1024.0 * 1024.0 * 1024.0);
        std::wstringstream ws;
        ws << std::fixed << std::setprecision(2) << gb << L" ГБ";
        return ws.str();
    }
}

LRESULT CALLBACK DialogProc(HWND hDlg, UINT message, WPARAM
wParam, LPARAM lParam) {
    switch (message) {
    case WM_CREATE: {
        HFONT hFont =
reinterpret_cast<HFONT>(GetStockObject(DEFAULT_GUI_FONT));

        CreateWindowW(L"STATIC", L"Название документа:", WS_CHILD |
WS_VISIBLE,
                      10, 10, 150, 25, hDlg, NULL, GetModuleHandle(NULL), NULL);
        HWND hNameEdit = CreateWindowW(L"EDIT", L"", WS_CHILD |
WS_VISIBLE | WS_BORDER | WS_TABSTOP,
                      170, 10, 250, 25, hDlg, (HMENU)IDC_NAME_EDIT,
GetModuleHandle(NULL), NULL);
        if (hFont) SendMessage(hNameEdit, WM_SETFONT, (WPARAM)hFont,
TRUE);

        CreateWindowW(L"STATIC", L"Автор:", WS_CHILD | WS_VISIBLE,
                      10, 45, 150, 25, hDlg, NULL, GetModuleHandle(NULL), NULL);
        HWND hAuthorEdit = CreateWindowW(L"EDIT", L"", WS_CHILD |
WS_VISIBLE | WS_BORDER | WS_TABSTOP,
                      170, 45, 250, 25, hDlg, (HMENU)IDC_AUTHOR_EDIT,
GetModuleHandle(NULL), NULL);
        if (hFont) SendMessage(hAuthorEdit, WM_SETFONT, (WPARAM)hFont,
TRUE);

        CreateWindowW(L"STATIC", L"Дата (ДД.ММ.ГГГГ):", WS_CHILD |
WS_VISIBLE,
```

```
            10, 80, 150, 25, hDlg, NULL, GetModuleHandle(NULL), NULL);
        HWND hDateEdit = CreateWindowW(L"EDIT", L"", WS_CHILD |
WS_VISIBLE | WS_BORDER | WS_TABSTOP,
            170, 80, 250, 25, hDlg, (HMENU)IDC_DATE_EDIT,
GetModuleHandle(NULL), NULL);
        if (hFont) SendMessage(hDateEdit, WM_SETFONT, (WPARAM)hFont,
TRUE);

        CreateWindowW(L"STATIC", L"Путь к файлу:", WS_CHILD |
WS_VISIBLE,
            10, 115, 150, 25, hDlg, NULL, GetModuleHandle(NULL), NULL);
        HWND hPathEdit = CreateWindowW(L"EDIT", L"", WS_CHILD |
WS_VISIBLE | WS_BORDER | WS_TABSTOP,
            170, 115, 200, 25, hDlg, (HMENU)IDC_PATH_EDIT,
GetModuleHandle(NULL), NULL);
        if (hFont) SendMessage(hPathEdit, WM_SETFONT, (WPARAM)hFont,
TRUE);

        CreateWindowW(L"BUTTON", L"Обзор...", WS_CHILD | WS_VISIBLE |
WS_TABSTOP | BS_PUSHBUTTON,
            380, 115, 80, 25, hDlg, (HMENU)IDC_BROWSE_BUTTON,
GetModuleHandle(NULL), NULL);

        CreateWindowW(L"BUTTON", L"OK", WS_CHILD | WS_VISIBLE |
WS_TABSTOP | BS_DEFPUSHBUTTON,
            100, 160, 80, 30, hDlg, (HMENU)IDC_OK_BUTTON,
GetModuleHandle(NULL), NULL);
        CreateWindowW(L"BUTTON", L"Отмена", WS_CHILD | WS_VISIBLE |
WS_TABSTOP,
            220, 160, 80, 30, hDlg, (HMENU)IDC_CANCEL_BUTTON,
GetModuleHandle(NULL), NULL);

        if (isEditMode && editIndex >= 0) {
            Document doc;
            if (storage.isInSearchMode()) {
                doc = storage.getDocument(editIndex);
            } else {
                doc = storage.getDocumentFromStorage(editIndex);
            }
            SetDlgItemTextW(hDlg, IDC_NAME_EDIT, doc.getName().c_str());
            SetDlgItemTextW(hDlg, IDC_AUTHOR_EDIT, doc.getAuthor().c_str());
            SetDlgItemTextW(hDlg, IDC_DATE_EDIT, doc.getDate().c_str());
            SetDlgItemTextW(hDlg, IDC_PATH_EDIT, doc.getFilePath().c_str());
            SetWindowTextW(hDlg, L"Редактировать документ");
```

```
        } else {
            SetWindowTextW(hDlg, L"Добавить документ");
        }

        return 0;
    }
    case WM_COMMAND: {
        int wmId = LOWORD(wParam);
        if (wmId == IDC_BROWSE_BUTTON) {
            OPENFILENAMEW ofn;
            wchar_t szFile[MAX_PATH] = {0};

            ZeroMemory(&ofn, sizeof(ofn));
            ofn.lStructSize = sizeof(ofn);
            ofn.hwndOwner = hDlg;
            ofn.lpstrFile = szFile;
            ofn.nMaxFile = sizeof(szFile)/sizeof(szFile[0]);
            ofn.lpstrFilter = L"Все файлы\0*.*\0\0";
            ofn.nFilterIndex = 1;
            ofn.lpstrFileTitle = NULL;
            ofn.nMaxFileTitle = 0;
            ofn.lpstrInitialDir = NULL;
            ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST |
OFN_NOCHANGEDIR;

            if (GetOpenFileNameW(&ofn)) {
                SetDlgItemTextW(hDlg, IDC_PATH_EDIT, szFile);
            }
        }
        else if (wmId == IDC_OK_BUTTON) {
            wchar_t buffer[512];

            GetDlgItemTextW(hDlg, IDC_NAME_EDIT, buffer, 512);
            std::wstring name(buffer);
            GetDlgItemTextW(hDlg, IDC_AUTHOR_EDIT, buffer, 512);
            std::wstring author(buffer);
            GetDlgItemTextW(hDlg, IDC_DATE_EDIT, buffer, 512);
            std::wstring date(buffer);
            GetDlgItemTextW(hDlg, IDC_PATH_EDIT, buffer, 512);
            std::wstring path(buffer);

            if (ValidateDocumentFields(name, path)) {
                Document newDoc(name, author, date, path);
```

```
            if (isEditMode && editIndex >= 0) {
                // Определяем, нужно ли редактировать по индексу из поиска или основного списка
                if (storage.isInSearchMode()) {
                    // Получаем документ из результатов поиска
                    Document oldDoc = storage.getDocument(editIndex);
                    // Находим его позицию в основном списке
                    const std::vector<Document>& allDocs = storage.getCurrentDocuments();
                    auto it = std::find(allDocs.begin(), allDocs.end(), oldDoc);
                    if (it != allDocs.end()) {
                        size_t realIndex = std::distance(allDocs.begin(), it);
                        storage.editDocument(realIndex, newDoc);
                    }
                } else {
                    storage.editDocument(editIndex, newDoc);
                }
            } else {
                storage.addDocument(newDoc);
            }

            // Если был активен поиск, обновляем результаты
            if (storage.isInSearchMode()) {
                storage.searchDocuments(storage.getLastSearchQuery());
            }

            RefreshListView(hListView);
        } else {
            return 0;
        }

        DestroyWindow(hDlg);
        hDialog = NULL;
    } else if (wmId == IDC_CANCEL_BUTTON) {
        DestroyWindow(hDlg);
        hDialog = NULL;
    }
    break;
}
case WM_CLOSE:
    DestroyWindow(hDlg);
    hDialog = NULL;
    break;
case WM_DESTROY:
```

```
        hDialog = NULL;
        break;
    }
    return DefWindowProcW(hDlg, message, wParam, lParam);
}

void ShowDocumentDialog(HWND hParent, bool edit, int index) {
    if (hDialog != NULL) {
        SetForegroundWindow(hDialog);
        return;
    }

    isEditMode = edit;
    editIndex = index;

    WNDCLASSEXW wc;
    ZeroMemory(&wc, sizeof(WNDCLASSEXW));
    wc.cbSize = sizeof(WNDCLASSEXW);
    wc.style = CS_HREDRAW | CS_VREDRAW;
    wc.lpfnWndProc = DialogProc;
    wc.hInstance = GetModuleHandle(NULL);
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wc.lpszClassName = L"DocumentDialogClass";

    static bool classRegistered = false;
    if (!classRegistered) {
        if (!RegisterClassExW(&wc)) {
            MessageBoxW(NULL, L"Не удалось зарегистрировать класс диалога",
                    L"Ошибка", MB_OK | MB_ICONERROR);
            return;
        }
        classRegistered = true;
    }

    hDialog = CreateWindowExW(0, L"DocumentDialogClass", L"",
                        WS_POPUP | WS_CAPTION | WS_SYSMENU,
                        0, 0, 480, 230, hParent, NULL, GetModuleHandle(NULL),
NULL);

    if (!hDialog) {
        MessageBoxW(NULL, L"Не удалось создать диалоговое окно",
                L"Ошибка", MB_OK | MB_ICONERROR);
        return;
```

```
    }

    RECT rcParent, rcDlg;
    GetWindowRect(hParent, &rcParent);
    GetWindowRect(hDialog, &rcDlg);

    int x = rcParent.left + (rcParent.right - rcParent.left - (rcDlg.right - rcDlg.left)) /
2;
    int y = rcParent.top + (rcParent.bottom - rcParent.top - (rcDlg.bottom -
rcDlg.top)) / 2;
    SetWindowPos(hDialog, NULL, x, y, 0, 0, SWP_NOSIZE |
SWP_NOZORDER);

    ShowWindow(hDialog, SW_SHOW);
    UpdateWindow(hDialog);
}

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
LPSTR lpCmdLine, int nCmdShow) {
    INITCOMMONCONTROLSEX icex;
    icex.dwSize = sizeof(INITCOMMONCONTROLSEX);
    icex.dwICC = ICC_LISTVIEW_CLASSES | ICC_WIN95_CLASSES;
    InitCommonControlsEx(&icex);

    WNDCLASSEXW wc;
    ZeroMemory(&wc, sizeof(WNDCLASSEXW));
    wc.cbSize = sizeof(WNDCLASSEXW);
    wc.style = CS_HREDRAW | CS_VREDRAW;
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInstance;
    wc.hIcon = LoadIcon(NULL, IDI_APPLICATION);
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_WINDOW + 1);
    wc.lpszMenuName = NULL;
    wc.lpszClassName = L"DocStorageClass";
    wc.hIconSm = LoadIcon(NULL, IDI_APPLICATION);

    if (!RegisterClassExW(&wc)) {
        MessageBoxW(NULL, L"Не удалось зарегистрировать класс окна",
                L"Ошибка", MB_OK | MB_ICONERROR);
        return 0;
    }

    hMainWindow = CreateWindowW(L"DocStorageClass", L"Хранилище
```

```
проектной документации",
                        WS_OVERLAPPEDWINDOW | WS_VISIBLE,
                        CW_USEDEFAULT, CW_USEDEFAULT,
                        900, 600, NULL, NULL, hInstance, NULL);

    if (!hMainWindow) {
        MessageBoxW(NULL, L"Не удалось создать главное окно",
                L"Ошибка", MB_OK | MB_ICONERROR);
        return 0;
    }

    ShowWindow(hMainWindow, nCmdShow);
    UpdateWindow(hMainWindow);

    MSG msg;
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    return (int)msg.wParam;
}

LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM
wParam, LPARAM lParam) {
    static HWND hSearchLabel; // Добавляем переменную для хранения метки
поиска

    switch (msg) {
    case WM_CREATE: {
        HFONT hFont =
reinterpret_cast<HFONT>(GetStockObject(DEFAULT_GUI_FONT));

        hListView = CreateWindowW(L"SysListView32", L"",
                            WS_CHILD | WS_VISIBLE | WS_BORDER |
                            LVS_REPORT | LVS_SINGLESEL |
LVS_SHOWSELALWAYS,
                            10, 50, 860, 400, hwnd,
                            (HMENU)IDC_LISTVIEW, GetModuleHandle(NULL),
NULL);
        if (hFont) SendMessage(hListView, WM_SETFONT, (WPARAM)hFont,
TRUE);
        InitListView(hListView);
```

```
    // Первый ряд кнопок
    HWND hAddBtn = CreateWindowW(L"BUTTON", L"Добавить",
                        WS_CHILD | WS_VISIBLE | WS_TABSTOP |
BS_PUSHBUTTON,
                        10, 10, 100, 30, hwnd,
                        (HMENU)IDC_ADDBUTTON, NULL, NULL);
    if (hFont) SendMessage(hAddBtn, WM_SETFONT, (WPARAM)hFont,
TRUE);

    HWND hEditBtn = CreateWindowW(L"BUTTON", L"Редактировать",
                         WS_CHILD | WS_VISIBLE | WS_TABSTOP |
BS_PUSHBUTTON,
                         120, 10, 120, 30, hwnd,
                         (HMENU)IDC_EDITBUTTON, NULL, NULL);
    if (hFont) SendMessage(hEditBtn, WM_SETFONT, (WPARAM)hFont,
TRUE);

    HWND hDeleteBtn = CreateWindowW(L"BUTTON", L"Удалить",
                          WS_CHILD | WS_VISIBLE | WS_TABSTOP |
BS_PUSHBUTTON,
                          250, 10, 100, 30, hwnd,
                          (HMENU)IDC_DELETEBUTTON, NULL, NULL);
    if (hFont) SendMessage(hDeleteBtn, WM_SETFONT, (WPARAM)hFont,
TRUE);

    HWND hLoadBtn = CreateWindowW(L"BUTTON", L"Загрузить",
                         WS_CHILD | WS_VISIBLE | WS_TABSTOP |
BS_PUSHBUTTON,
                         360, 10, 100, 30, hwnd,
                         (HMENU)IDC_LOADBUTTON, NULL, NULL);
    if (hFont) SendMessage(hLoadBtn, WM_SETFONT, (WPARAM)hFont,
TRUE);

    HWND hSaveBtn = CreateWindowW(L"BUTTON", L"Сохранить",
                         WS_CHILD | WS_VISIBLE | WS_TABSTOP |
BS_PUSHBUTTON,
                         470, 10, 100, 30, hwnd,
                         (HMENU)IDC_SAVEBUTTON, NULL, NULL);
    if (hFont) SendMessage(hSaveBtn, WM_SETFONT, (WPARAM)hFont,
TRUE);

    HWND hClearBtn = CreateWindowW(L"BUTTON", L"Очистить все",
                         WS_CHILD | WS_VISIBLE | WS_TABSTOP |
BS_PUSHBUTTON,
```

```
                        580, 10, 110, 30, hwnd,
                        (HMENU)IDC_CLEARBUTTON, NULL, NULL);
    if (hFont) SendMessage(hClearBtn, WM_SETFONT, (WPARAM)hFont,
TRUE);

    // Второй ряд: поиск
    hSearchLabel = CreateWindowW(L"STATIC", L"Поиск:", WS_CHILD |
WS_VISIBLE,
            10, 460, 50, 25, hwnd, (HMENU)IDC_STATIC,
GetModuleHandle(NULL), NULL);
    if (hFont) SendMessage(hSearchLabel, WM_SETFONT, (WPARAM)hFont,
TRUE);

    hSearchEdit = CreateWindowW(L"EDIT", L"",
                        WS_CHILD | WS_VISIBLE | WS_BORDER |
WS_TABSTOP | ES_AUTOHSCROLL,
                        65, 460, 200, 25, hwnd,
                        (HMENU)IDC_SEARCHEDIT, NULL, NULL);
    if (hFont) SendMessage(hSearchEdit, WM_SETFONT, (WPARAM)hFont,
TRUE);

    HWND hSearchBtn = CreateWindowW(L"BUTTON", L"Найти",
                        WS_CHILD | WS_VISIBLE | WS_TABSTOP |
BS_PUSHBUTTON,
                        275, 460, 80, 25, hwnd,
                        (HMENU)IDC_SEARCHBUTTON, NULL, NULL);
    if (hFont) SendMessage(hSearchBtn, WM_SETFONT, (WPARAM)hFont,
TRUE);

    HWND hClearSearchBtn = CreateWindowW(L"BUTTON", L"Очистить
поиск",
                        WS_CHILD | WS_VISIBLE | WS_TABSTOP |
BS_PUSHBUTTON,
                        365, 460, 120, 25, hwnd,
                        (HMENU)IDC_CLEARSEARCHBUTTON, NULL,
NULL);
    if (hFont) SendMessage(hClearSearchBtn, WM_SETFONT,
(WPARAM)hFont, TRUE);

    RefreshListView(hListView);
    break;
  }
  case WM_SIZE: {
    int width = LOWORD(lParam);
```

```
        int height = HIWORD(lParam);

        // Изменяем размер ListView
        MoveWindow(hListView, 10, 50, width - 30, height - 140, TRUE);

        // Перемещаем элементы поиска
        MoveWindow(hSearchLabel, 10, height - 80, 50, 25, TRUE);
        MoveWindow(hSearchEdit, 65, height - 80, 200, 25, TRUE);
        MoveWindow(GetDlgItem(hwnd, IDC_SEARCHBUTTON), 275, height -
80, 80, 25, TRUE);
        MoveWindow(GetDlgItem(hwnd, IDC_CLEARSEARCHBUTTON), 365,
height - 80, 120, 25, TRUE);
        break;
    }
    case WM_COMMAND: {
        int wmId = LOWORD(wParam);

        if (wmId == IDC_ADDBUTTON) {
            ShowDocumentDialog(hwnd, false, -1);
        }
        else if (wmId == IDC_EDITBUTTON) {
            int index = ListView_GetNextItem(hListView, -1, LVNI_SELECTED);
            if (index >= 0) {
                ShowDocumentDialog(hwnd, true, index);
            } else {
                MessageBoxW(hwnd, L"Выберите документ для редактирования",
                        L"Информация", MB_OK | MB_ICONINFORMATION);
            }
        }
        else if (wmId == IDC_DELETEBUTTON) {
            int index = ListView_GetNextItem(hListView, -1, LVNI_SELECTED);
            if (index >= 0) {
                std::wstring docName = storage.getDocument(index).getName();
                std::wstring message = L"Удалить документ \"" + docName + L"\"?";

                if (MessageBoxW(hwnd, message.c_str(),
                            L"Подтверждение", MB_YESNO | MB_ICONQUESTION)
== IDYES) {
                    if (storage.deleteDocument(index)) {
                        RefreshListView(hListView);
                        MessageBoxW(hwnd, L"Документ успешно удален",
                                L"Успех", MB_OK | MB_ICONINFORMATION);
                    }
                }
```

```
        } else {
            MessageBoxW(hwnd, L"Выберите документ для удаления",
                    L"Информация", MB_OK | MB_ICONINFORMATION);
        }
    }
    else if (wmId == IDC_LOADBUTTON) {
        size_t beforeCount = storage.getTotalDocumentCount();
        bool mergeData = false;

        if (beforeCount > 0) {
            int result = MessageBoxW(hwnd,
                L"Выберите вариант загрузки:\n\n"
                L"Да - добавить к существующим документам\n"
                L"Нет - заменить все документы\n"
                L"Отмена - отменить загрузку",
                L"Загрузка документов",
                MB_YESNOCANCEL | MB_ICONQUESTION);

            if (result == IDCANCEL) break;
            mergeData = (result == IDYES);
        }

        if (!FileExists(L"documents.txt")) {
            if (MessageBoxW(hwnd, L"Файл documents.txt не найден.\n"
                        L"Создать пустой файл?",
                        L"Файл не найден",
                        MB_YESNO | MB_ICONQUESTION) == IDYES) {
                std::ofstream file("documents.txt", std::ios::binary);
                if (file.is_open()) {
                    unsigned char bom[] = {0xEF, 0xBB, 0xBF};
                    file.write(reinterpret_cast<const char*>(bom), sizeof(bom));
                    file.close();
                    MessageBoxW(hwnd, L"Создан пустой файл documents.txt",
                            L"Информация", MB_OK | MB_ICONINFORMATION);
                }
            }
            break;
        }

        if (storage.loadFromFile(L"documents.txt", !mergeData)) {
            size_t afterCount = storage.getTotalDocumentCount();
            RefreshListView(hListView);

            std::wstring message;
```

```
            if (mergeData) {
                size_t added = afterCount - beforeCount;
                message = L"Добавлено " + std::to_wstring(added) +
                        L" новых документов\n" +
                        L"Всего документов: " + std::to_wstring(afterCount);
            } else {
                message = L"Загружено " + std::to_wstring(afterCount) + L"
документов";
            }
            MessageBoxW(hwnd, message.c_str(),
                    L"Загрузка завершена", MB_OK | MB_ICONINFORMATION);
        } else {
            MessageBoxW(hwnd, L"Ошибка при чтении файла documents.txt\n"
                    L"Файл может быть поврежден или иметь неверный
формат.",
                    L"Ошибка загрузки", MB_OK | MB_ICONERROR);
        }
    }
    else if (wmId == IDC_SAVEBUTTON) {
        size_t count = storage.getTotalDocumentCount();
        if (count == 0) {
            if (MessageBoxW(hwnd, L"Нет документов для сохранения.\n"
                    L"Создать пустой файл?",
                    L"Предупреждение", MB_YESNO | MB_ICONQUESTION)
== IDYES) {
                std::ofstream file("documents.txt", std::ios::binary);
                if (file.is_open()) {
                    unsigned char bom[] = {0xEF, 0xBB, 0xBF};
                    file.write(reinterpret_cast<const char*>(bom), sizeof(bom));
                    file.close();
                    MessageBoxW(hwnd, L"Создан пустой файл documents.txt",
                            L"Информация", MB_OK | MB_ICONINFORMATION);
                }
            }
            break;
        }

        if (FileExists(L"documents.txt")) {
            std::wstring fileInfo = GetFileSizeString(L"documents.txt");
            if (MessageBoxW(hwnd, (L"Файл documents.txt уже существует.\n" +
                    fileInfo + L"\n" +
                    L"Перезаписать файл?").c_str(),
                    L"Подтверждение",
                    MB_YESNO | MB_ICONQUESTION) != IDYES) {
```

```
            break;
        }
    }

    if (storage.saveToFile(L"documents.txt")) {
        std::wstring message = L"Успешно сохранено " +
std::to_wstring(count) +
                               L" документов в documents.txt";
        MessageBoxW(hwnd, message.c_str(),
                    L"Сохранение завершена", MB_OK |
MB_ICONINFORMATION);
    } else {
        MessageBoxW(hwnd, L"Ошибка при сохранении в documents.txt\n"
                     L"Проверьте права доступа к файлу.",
                    L"Ошибка сохранения", MB_OK | MB_ICONERROR);
    }
}
else if (wmId == IDC_CLEARBUTTON) {
    size_t count = storage.getTotalDocumentCount();
    if (count == 0) {
        MessageBoxW(hwnd, L"Хранилище уже пустое",
                    L"Информация", MB_OK | MB_ICONINFORMATION);
        break;
    }

    std::wstring message = L"Вы действительно хотите удалить ВСЕ
документы?\n"
                           L"Всего документов: " + std::to_wstring(count) +
                           L"\n\nЭта операция необратима!";

    if (MessageBoxW(hwnd, message.c_str(),
                    L"Подтверждение удаления",
                    MB_YESNO | MB_ICONWARNING) == IDYES) {
        storage.clearAllDocuments();
        RefreshListView(hListView);
        MessageBoxW(hwnd, L"Все документы успешно удалены",
                    L"Очистка завершена", MB_OK | MB_ICONINFORMATION);
    }
}
else if (wmId == IDC_SEARCHBUTTON) {
    wchar_t buffer[256];
    GetWindowTextW(hSearchEdit, buffer, 256);
    std::wstring keyword = TrimWString(std::wstring(buffer));
```

```
            if (keyword.empty()) {
                // Если поле поиска пустое, сбрасываем поиск
                storage.clearSearch();
                RefreshListView(hListView);
                MessageBoxW(hwnd, L"Поле поиска пустое. Показаны все
документы.",
                        L"Поиск", MB_OK | MB_ICONINFORMATION);
            } else {
                storage.searchDocuments(keyword);
                RefreshListView(hListView);

                if (storage.getDocumentCount() == 0) {
                    MessageBoxW(hwnd, L"Документы не найдены",
                            L"Поиск", MB_OK | MB_ICONINFORMATION);
                }
            }
        }
        else if (wmId == IDC_CLEARSEARCHBUTTON) {
            SetWindowTextW(hSearchEdit, L"");
            storage.clearSearch();
            RefreshListView(hListView);
        }
        break;
    }
    case WM_NOTIFY: {
        LPNMHDR nmhdr = (LPNMHDR)lParam;
        if (nmhdr->idFrom == IDC_LISTVIEW && nmhdr->code ==
NM_DBLCLK) {
            int index = ListView_GetNextItem(hListView, -1, LVNI_SELECTED);
            if (index >= 0) {
                ShowDocumentDialog(hwnd, true, index);
            }
        }
        break;
    }
    case WM_DESTROY:
        PostQuitMessage(0);
        break;
    default:
        return DefWindowProcW(hwnd, msg, wParam, lParam);
    }
    return 0;
}
```

Приложение Б. Диаграмма последовательностей

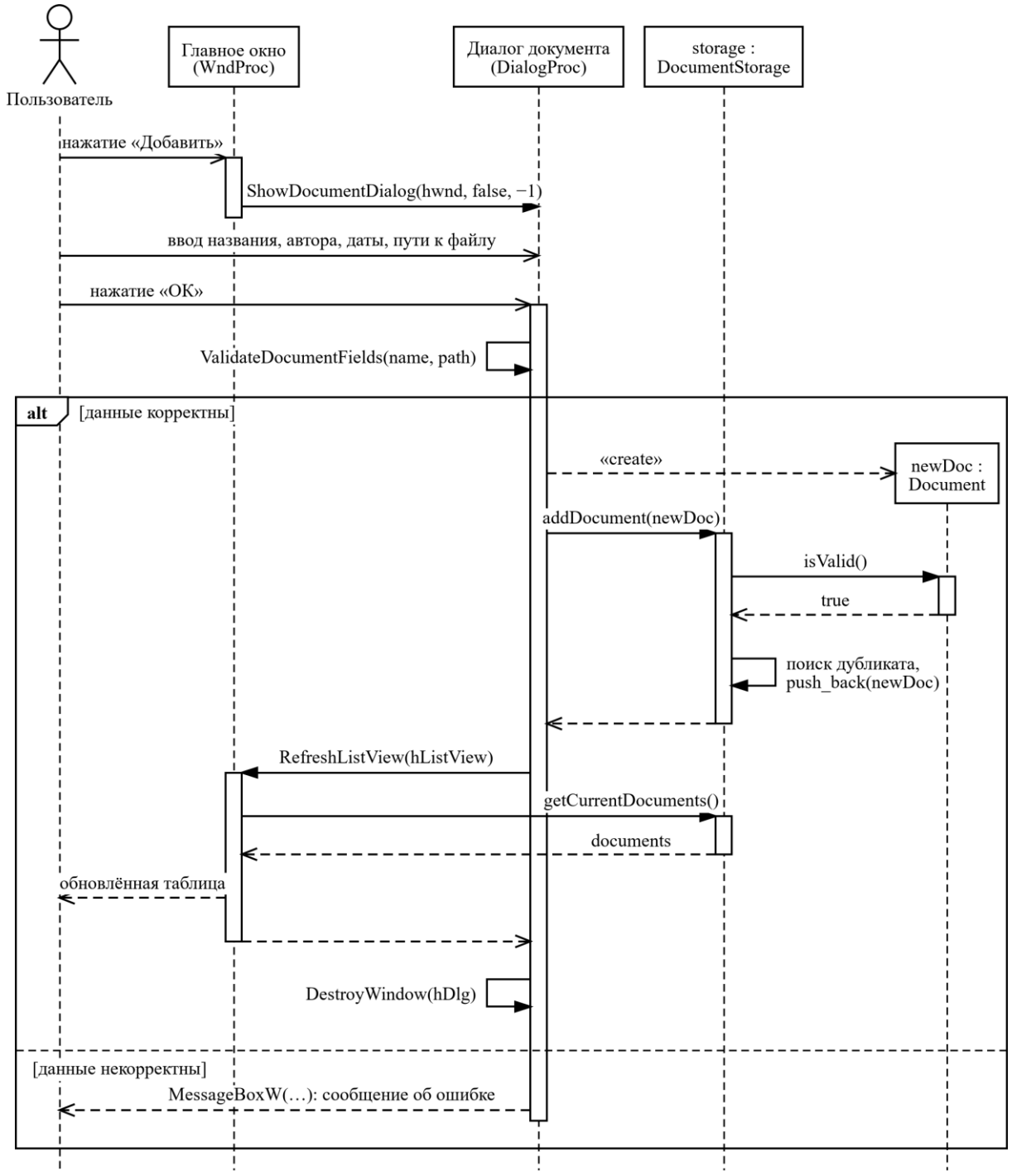


Рисунок Б.1 – Диаграмма последовательности добавления документа

Приложение В. Тестовые данные

```
Проект жилого дома;Иванов Иван Иванович;15.01.2023;C:\Projects\house.pdf
Техническое задание;Петров Петр
Петрович;20.02.2023;C:\Docs\specification.docx
Смета расходов;Сидоров Сидор
Сидорович;10.03.2023;C:\Finance\estimate.xlsx
```

```
Архитектурный план;Васильев Василий Васильевич;25.03.2023;C:\Plans\architecture.dwg
Инженерные сети;Кузнецов Алексей Николаевич;05.04.2023;C:\Engineering\networks.pdf
Ландшафтный дизайн;Смирнова Ольга Владимировна;15.04.2023;C:\Design\landscape.jpg
```

Приложение Г. Руководство по установке

1. **Требования к системе**:
   - Windows 7 или выше
   - 50 MB свободного места
   - Дополнительные библиотеки не требуются (статическая сборка)
2. **Установка**:
   - Распаковать архив в любую папку
   - Запустить DocStorage.exe
   - При первом запуске создастся файл documents.txt
3. **Обновление**:
   - Закрыть программу
   - Скопировать новые файлы
   - Запустить программу